

P1729R5

Text Parsing

Published Proposal, 2024-10-15

This version:

<http://wg21.link/P1729R5>

Authors:

[Elias Kosunen](#)

[Victor Zverovich](#)

Audience:

LEWG, SG16

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper discusses a new text parsing facility to complement the text formatting functionality of `std::format`, proposed in [\[P0645\]](#).

Table of Contents

1 Revision history

- 1.1 Changes since R4
- 1.2 Changes since R3
- 1.3 Changes since R2
- 1.4 Changes since R1

2 Introduction

3 Examples

- 3.1 Basic example

- 3.2 Reading multiple values at once
- 3.3 Reading from a range
- 3.4 Reading multiple values in a loop
- 3.5 Alternative error handling
- 3.6 Scanning a user-defined type

4 Design

- 4.1 Overview
- 4.2 Format strings
- 4.3 Format string specifiers
- 4.4 Ranges
- 4.5 Argument passing, and return type of `scan`
- 4.6 Error handling
- 4.7 Binary footprint and type erasure
- 4.8 Safety
- 4.9 Extensibility
- 4.10 Locales
- 4.11 Encoding
- 4.12 Performance
- 4.13 Integration with chrono
- 4.14 Impact on existing code

5 Existing work

6 Future extensions

- 6.1 Integration with `stdio`
- 6.2 `scanf`-like `[character set]` matching
- 6.3 Reading code points (or even grapheme clusters?)
- 6.4 Reading strings and chars of different width
- 6.5 Scanning of ranges
- 6.6 Default values for scanned values
- 6.7 Assignment suppression / discarding values

7 Specification

- 7.1 General
- 7.2 Scanning `[scan]`

References

Informative References

§ 1. Revision history

§ 1.1. Changes since R4

- Include preliminary wording
- Revamp argument handling and erasure machinery to better elide copies and moves
 - Redefine `make_scan_result`, `make_scan_args`, and `scan-arg-store`, add `fill_scan_result`.
- Add discussion on the name `scan`.
- Make fill+align logic more lenient and easy to understand.
- Add precision-specifier to specify maximum field width.
 - Modify width-specifier to specify minimum field width.
- Remove `scan_error` success state, replace with `expected<void, scan_error>`.
- Split `scan_error::value_out_of_range` into a four separate enumerators, for positive and negative overflow and underflow.
- Add `scan_error::invalid_literal`, `scan_error::invalid_fill`, and `scan_error::length_too_short`, all of which were previously covered by `scan_error::invalid_scanned_value`.
- Revise error handling in `scanner::parse`.
 - `scanner::parse` now returns `iterator`, instead of `expected<iterator, scan_error>`.
 - Add `scan_format_string_error`.
- Rename `scan_error::end_of_range` -> `scan_error::end_of_input`.
- Add parsing of pointers (`void*` and `const void*`).
- Remove requirement for localized numbers to have "correct" digit grouping as specified by `numpunct::grouping`.
- Remove design discussion on a dedicated flag for thousands separators (`'`), separate from locale.
- Remove detailed design discussion on error handling alternatives.

- Update example on user-defined type scanning.
- Clarify meaning of "whitespace" further in § 4.2 [Format strings](#).
- Fix example claiming `std::expected::operator->` throws on an expected containing an error.
- SG9: Make `borrowed_tail_subrange_t` exposition-only (*borrowed-tail-subrange-t*).
- Make concept `scannable_range` exposition-only (*scannable-range*).
 - SG9: Add requirement to *scannable-range* for the `value_type` to either be `char` or `wchar_t`.
- Formatting and styling fixes.

§ 1.2. Changes since R3

- Replace `scan_args_for` with `scan_args` and `wscan_args` for consistency with `std::format`.
- Rename `borrowed_ssubrange_t` to `borrowed_tail_subrange_t` partly based on the naming from ranges-v3 (`tail_view`).
- Replace `format_string` with `scan_format_string`, with a `Range` template parameter.
 - Enables compile-time checking for compatibility of the source range, and arguments to scan
- Make `[v]scan_result_type` (the return types of `std::scan` and `std::vscan`) exposition only.
- Remove `visit_scan_arg`: follow [P2637] and use `std::variant::visit`, instead.
- Add discussion on `stdin` support, guided by SG9 polls.
- Make encoding errors be errors for strings, instead of garbage-in-garbage-out.
- Add further discussion on field widths.
- Add example as rationale for mandating `forward_range`.

§ 1.3. Changes since R2

- Return a `subrange` from `scan`, instead of just an iterator: discussion in § 4.5 [Argument passing, and return type of scan](#).

- Default `CharT` to `char` in `scanner` for consistency with `formatter` (previously no default for `CharT`).
- Add design discussion about thousands separators.
- Add design discussion about additional error information.
- Add clarification about field width calculation in [§ 4.3.4 Width and precision](#).
- Add note about scope at the end of [§ 2 Introduction](#).
- Fix/clarify error handling in example [§ 3.5 Alternative error handling](#).
- Address SG16 feedback:
 - Add definition of "whitespace", and clarify matching of non-whitespace literal characters, in [§ 4.2 Format strings](#).
 - Add section about text encoding [§ 4.11 Encoding](#), and an example about handling reading code units [§ 4.3.8 Type specifiers: CharT](#).
 - Add example about using locales in [§ 4.10 Locales](#).
 - Add potential future extension: [§ 6.3 Reading code points \(or even grapheme clusters?\)](#)

§ 1.4. Changes since R1

- Thoroughly describe the design
- Add examples
- Add specification (synopses only)
- Design changes:
 - Return an `expected` containing a `tuple` from `std::scan`, instead of using output parameters
 - Make `std::scan` take a range instead of a `string_view`
 - Remove support for partial successes

§ 2. Introduction

With the introduction of `std::format` [P0645], standard C++ has a convenient, safe, performant, extensible, and elegant facility for text formatting, over `std::ostream` and the `printf`-family of functions. The story is different for simple text parsing: the standard only provides `std::istream`

and the `scanf` family, both of which have issues. This asymmetry is also arguably an inconsistency in the standard library.

According to [\[CODESEARCH\]](#), a C and C++ codesearch engine based on the ACTCD19 dataset, there are 389,848 calls to `sprintf` and 87,815 calls to `sscanf` at the time of writing. So although formatted input functions are less popular than their output counterparts, they are still widely used.

The lack of a general-purpose parsing facility based on format strings has been raised in [\[P1361\]](#) in the context of formatting and parsing of dates and times.

This paper proposes adding a symmetric parsing facility, `std::scan`, to complement `std::format`. This facility is based on the same design principles and shares many features with `std::format`.

This facility is not a parser per se, as it is probably not sufficient for parsing something more complicated, e.g. JSON. This is not a parser combinator library. This is intended to be an almost-drop-in replacement for `sscanf`, capable of being a building block for a more complicated parser.

§ 3. Examples

§ 3.1. Basic example

```
if (auto result = std::scan<std::string, int>("answer = 42", "{} = {}")) {  
    //          ~~~~~ output types      ~~~~~ input      ~~~~~  
    //          output types      input      format  
    //                                     string  
  
    const auto& [key, value] = result->values();  
    //          ~~~~~  
    //          scanned  
    //          values  
  
    // result is a std::expected<std::scan_result<...>>.  
    // result->range() gives an empty range.  
    // result->begin() == result->end()  
    // key == "answer"  
    // value == 42  
} else {  
    // We would end up here if we had an error.  
    std::scan_error error = result.error();  
}
```

§ 3.2. Reading multiple values at once

```
auto input = "25 54.32E-1 Thompson 56789 0123";

auto result = std::scan<int, float, string_view, int, float, int>(
    input, "{:d}{:f}{:9}{:2i}{:g}{:o}");

// result is a std::expected, value() will throw if it doesn't contain a value
auto [i, x, str, j, y, k] = result.value().values();

// i == 25
// x == 54.32e-1
// str == "Thompson"
// j == 56
// y == 789.0
// k == 0123
```

§ 3.3. Reading from a range

```
std::string input{"123 456"};
if (auto result = std::scan<int>(std::views::reverse(input), "{}")) {
    // If only a single value is returned, it can be accessed with result->value()
    // result->value() == 654
}
```

§ 3.4. Reading multiple values in a loop

```
std::vector<int> read_values;
std::ranges::forward_range auto range = ...;

auto input = std::ranges::subrange{range};

while (auto result = std::scan<int>(input, "{}")) {
    read_values.push_back(result->value());
    input = result->range();
}
```

§ 3.5. Alternative error handling

```
// Since std::scan returns a std::expected,
// its monadic interface can be used

auto result = std::scan<int>(..., "{}")
    .transform([](auto result) {
        return result.value();
    });
if (!result) {
    // handle error
}
int num = *result;

// With [P2561]:
int num = std::scan<int>(..., "{}").try?.value();
```

§ 3.6. Scanning a user-defined type

```
struct mytype {
    int a{}, b{};
};

// Specialize std::scanner to add support for user-defined types.
template <>
struct std::scanner<mytype> {
    // Parse format string: only accept empty format strings
    template <typename ParseContext>
    constexpr auto parse(ParseContext& pctx)
        -> typename ParseContext::iterator {
        return pctx.begin();
    }

    // Scan the value from `ctx`:
    // delegate to `std::scan`
    template <typename Context>
    auto scan(mytype& val, Context& ctx) const
        -> std::expected<typename Context::iterator, std::scan_error> {
        return std::scan<int, int>(ctx.range(), "[{}, {}]")
            .transform([&val](const auto& result) {
```



```

        std::tie(val.a, val.b) = result.values();
        return result.begin();
    });
}
};

auto result = std::scan<mytype>("[123, 456]", "{}");
// result->value().a == 123
// result->value().b == 456

```

§ 4. Design

The new parsing facility is intended to complement the existing C++ I/O streams library, integrate well with the chrono library, and provide an API similar to `std::format`. This section discusses the major features of its design.

§ 4.1. Overview

The main user-facing part of the library described in this paper, is the function template `std::scan`, the input counterpart of `std::format`. The signature of `std::scan` is as follows:

```

template <class... Args, scannable-range<char> Range>
auto scan(Range&& range, scan_format_string<Range, Args...> fmt)
    -> expected<scan_result<borrowed-tail-subrange-t<Range>, Args...>, scan_error>

template <class... Args, scannable-range<wchar_t> Range>
auto scan(Range&& range, wscan_format_string<Range, Args...> fmt)
    -> expected<scan_result<borrowed-tail-subrange-t<Range>, Args...>, scan_error>

```

`std::scan` reads values of type `Args...` from the `range` it's given, according to the instructions given to it in the format string, `fmt`. `std::scan` returns a `std::expected`, containing either a `scan_result`, or a `scan_error`. The `scan_result` object contains a `subrange` pointing to the unparsed input, and a `tuple` of `Args...`, containing the scanned values.

§ 4.1.1. Naming of the function `scan`

The proposed name for the function `std::scan` has caused some dissent, namely in the FP and HPC circles. They argue, that `scan` is the name of an algorithm, which is also already in the standard library, in the form of `std::inclusive_scan` and `std::exclusive_scan`: [Wikipedia: Prefix sum](https://en.cppreference.com/algorithm/scan) [cppreference.com: std::inclusive_scan](https://en.cppreference.com/algorithm/scan).

However, the aforementioned algorithm doesn't have exclusive ownership of the name `scan`. `scan` is an extremely common name for the operation proposed in this paper, and has very long-standing precedent in the C and C++ standard libraries in the form of the `scanf` family of functions.

An alternative often thrown around is the name `parse`. There are two problems with that name:

- `parse` is a larger land-grab than `scan`, and is potentially misleading. The facility proposed in this paper is NOT a parser combinator library, but something closer to a `scanf` replacement, with a more limited scope.
- `parse` is already a term used in this paper, and in `std::format`: it's used to describe the action of format string parsing. It's found in the member function `std::formatter::parse` / `std::scanner::parse`, and in the class templates `std::basic_format_parse_context` / `std::basic_scan_parse_context`. The member functions doing the actual formatting in `formatter` and `scanner` are called the same as the public interface functions: `format` and `scan`, respectively. Were `std::scan` be called `std::parse`, it's unclear what `std::scanner`, `std::scanner::parse`, `std::scanner::scan`, and `std::basic_scan_parse_context` should be called.

§ 4.2. Format strings

As with `printf`, the `scanf` syntax has the advantage of being familiar to many programmers. However, it has similar limitations:

- Many format specifiers like `hh`, `h`, `l`, `j`, etc. are used only to convey type information. They are redundant in type-safe parsing and would unnecessarily complicate specification and parsing.
- There is no standard way to extend the syntax for user-defined types.
- Using `'%'` in a custom format specifier poses difficulties, e.g. for `get_time`-like time parsing.

Therefore, we propose a syntax based on `std::format` and `[PARSE]`. This syntax employs `'{'` and `'}'` as replacement field delimiters instead of `'%'`. It will provide the following advantages:

- An easy-to-parse mini-language focused on the data format rather than conveying the type information
- Extensibility for user-defined types
- Positional arguments
- Support for both locale-specific and locale-independent parsing (see [§ 4.10 Locales](#))
- Consistency with `std::format`.

At the same time, most of the specifiers will remain quite similar to the ones in `scanf`, which can simplify a, possibly automated, migration.

Maintaining similarity with `scanf`, for any literal non-whitespace character in the format string, an identical character is consumed from the input range. For whitespace characters, all available whitespace characters are consumed.

In this proposal, "whitespace" is defined to be the Unicode code points with the `Pattern_White_Space` property, as defined by UAX #31 (UAX31-R3a). Those code points are:

- ASCII whitespace characters:
 - U+0009 (HORIZONTAL TABULATION `'\t'`)
 - U+000A (LINE FEED `'\n'`)
 - U+000B (VERTICAL TABULATION `'\v'`)
 - U+000C (FORM FEED `'\f'`)
 - U+000D (CARRIAGE RETURN `'\r'`)
 - U+0020 (SPACE `' '`)
- U+0085 (NEXT LINE)
- U+200E (LEFT-TO-RIGHT MARK)
- U+200F (RIGHT-TO-LEFT MARK)
- U+2028 (LINE SEPARATOR)
- U+2029 (PARAGRAPH SEPARATOR)

Unicode defines a lot of different things in the realm of whitespace, all for different kinds of use cases. The `Pattern_White_Space`-property is chosen for its stability (it's guaranteed to not change), and because its intended use is for classifying things that should be treated as whitespace in machine-readable syntaxes. `std::isspace` is insufficient for usage in a Unicode world, because it only accepts a single code unit as input.

EXAMPLE 1

```
auto r0 = std::scan<char>("abcd", "ab{}d"); // r0->value() == 'c'

auto r1 = std::scan<string, string>("abc \n def", "{} {}");
const auto& [s1, s2] = r1->values(); // s1 == "abc", s2 == "def"
```

As mentioned above, the format string syntax consists of replacement fields delimited by curly brackets (`{` and `}`). Each of these replacement fields corresponds to a value to be scanned from the input range. The replacement field syntax is quite similar to `std::format`, as can be seen below. Elements that are in one but not the other are highlighted. Note how the `scan` syntax is mostly a subset of the `format` syntax, except for the two added entries under *type*.

`scan` replacement field syntax

std-format-spec:

*fill-and-align*_{opt} *width*_{opt} *precision*_{opt} *L*_{opt} *type*_{opt}

fill-and-align:

*fill*_{opt} *align*

fill:

any character other than `{` or `}`

align: one of

`<` `>` `^`

width:

positive-integer

precision:

▪ *nonnegative-integer*

type: one of

`a` `A` `b` `B` `c` `d` `e` `E` `f` `F` `g` `G` `i` `o` `p` `P` `s` `u` `x` `X` `?`

`format` replacement field syntax

std-format-spec:

*fill-and-align*_{opt} *sign*_{opt} *#*_{opt} *0*_{opt} *width*_{opt} *precision*_{opt} *L*_{opt} *type*_{opt}

fill-and-align:

*fill*_{opt} *align*

fill:

any character other than { or }

align: one of

< > ^

sign: one of

+ - space

width:

positive-integer

{ *arg-id*_{opt} }

precision:

▪ *nonnegative-integer*

▪ { *arg-id*_{opt} }

type: one of

a A b B c d e E f F g G o p P s x X ?

NOTE: In addition to the list of presentation types above, [\[SCNLIB\]](#) also supports:

- [rNN](#), [RNN](#) for arbitrary-base integers (r/R stands for radix, as b/B is already taken)
- [U](#) for an Unicode code point
- [\[...\]](#) for scanf-like set of characters
- [/.../](#) for regex

These are currently not proposed. Some of these are mentioned in [§ 6 Future extensions](#).

§ 4.3. Format string specifiers

Below is a somewhat detailed description of each of the specifiers in a `std::scan` replacement field. This design attempts to maintain decent compatibility with `std::format` whenever practical, while also bringing in some ideas from `scanf`.

§ 4.3.1. Manual indexing

Like `std::format`, `std::scan` supports manual indexing of arguments in format strings. If manual indexing is used, all of the argument indices have to be spelled out. Different from `std::format`, the same index can only be used once.

EXAMPLE 2

```
auto r = std::scan<int, int, int>("0 1 2", "{1} {0} {2}");
auto [i0, i1, i2] = r->values();
// i0 == 1, i1 == 0, i2 == 2
```

§ 4.3.2. Fill and align

fill-and-align:

fill_{opt} align

fill:

any character other than { or }

align: one of

< > ^

The fill and align options are valid for all argument types. The fill character is denoted by the `fill`-option, or if it is absent, the space character ' '. The fill character can be any single Unicode scalar value. The field width is determined the same way as it is for `std::format`.

If an alignment is specified, the value to be parsed is assumed to be properly aligned with the specified fill character.

If a field width is specified, it will taken to be the minimum number of characters to be consumed from the input range. If a field precision is specified, it will taken to be the maximum number of characters to be consumed from the input range. If either field width or precision is specified, but no alignment is, the default alignment for the type is considered (see `std::format`).

For the '^' alignment, fill characters both before and after the value will be considered. The number of fill characters doesn't have to be equal: input will be parsed until either a non-fill character is encountered, or the (maximum) field precision is exhausted, after which checking is done for the (minimum) field width.

This spec is compatible with `std::format`, i.e., the same format string (wrt. fill and align) can be used with both `std::format` and `std::scan`, with round-trip semantics.

NOTE: For format type specifiers other than `'c'` (default for `char` and `wchar_t`, can be specified for `basic_string` and `basic_string_view`), leading whitespace is skipped regardless of alignment specifiers.

EXAMPLE 3

```

auto r0 = std::scan<int>(" 42", "{}"); // r0->value() == 42, r0->range
auto r1 = std::scan<char>(" x", "{}"); // r1->value() == ' ', r1->range
auto r2 = std::scan<char>("x  ", "{}"); // r2->value() == 'x', r2->range

auto r3 = std::scan<int>(" 42", "{:6}"); // r3->value() == 42, r3->range
auto r4 = std::scan<char>("x  ", "{:6}"); // r4->value() == 'x', r4->range

auto r5 = std::scan<int>("***42", "{:*>}"); // r5->value() == 42, r5->range
auto r6 = std::scan<int>("***42", "{:*>5}"); // r6->value() == 42, r6->range
auto r7 = std::scan<int>("***42", "{:*>4}"); // r7->value() == 42, r7->range
auto r8 = std::scan<int>("***42", "{:*>.4}"); // r8->value() == 4, r8->range
auto r9 = std::scan<int>("***42", "{:*>4.4}"); // r9->value() == 4, r9->range

auto r10 = std::scan<int>("42", "{:*>}"); // r10->value() == 42, r10->range
auto r11 = std::scan<int>("42", "{:*>5}"); // ERROR (length_too_short)
auto r12 = std::scan<int>("42", "{:*>.5}"); // r12->value() == 42, r12->range
auto r13 = std::scan<int>("42", "{:*>5.5}"); // ERROR (length_too_short)

auto r14 = std::scan<int>("42***", "{:*<}"); // r14->value() == 42, r14->range
auto r15 = std::scan<int>("42***", "{:*<5}"); // r15->value() == 42, r15->range
auto r16 = std::scan<int>("42***", "{:*<4}"); // r16->value() == 42, r16->range
auto r17 = std::scan<int>("42***", "{:*<.4}"); // r17->value() == 42, r17->range
auto r18 = std::scan<int>("42***", "{:*<4.4}"); // r18->value() == 42, r18->range

auto r19 = std::scan<int>("42", "{:*<}"); // r19->value() == 42, r19->range
auto r20 = std::scan<int>("42", "{:*<5}"); // ERROR (length_too_short)
auto r21 = std::scan<int>("42", "{:*<.5}"); // r21->value() == 42, r19->range
auto r22 = std::scan<int>("42", "{:*<5.5}"); // ERROR (length_too_short)

auto r23 = std::scan<int>("42", "{:*^}"); // r23->value() == 42, r23->range
auto r24 = std::scan<int>("*42*", "{:*^}"); // r24->value() == 42, r24->range
auto r25 = std::scan<int>("*42**", "{:*^}"); // r25->value() == 42, r25->range
auto r26 = std::scan<int>("**42*", "{:*^}"); // r26->value() == 42, r26->range

auto r27 = std::scan<int>("**42**", "{:*^6}"); // r27->value() == 42, r27->range
auto r28 = std::scan<int>("*42**", "{:*^5}"); // r28->value() == 42, r28->range
auto r29 = std::scan<int>("***42*", "{:*^5}"); // r29->value() == 42, r29->range
auto r30 = std::scan<int>("**42*", "{:*^6}"); // ERROR (length_too_short)
auto r31 = std::scan<int>("**42*", "{:*^6.6}"); // r31->value() == 42, r31->range
auto r32 = std::scan<int>("**42*", "{:*^6.6}"); // ERROR (length_too_short)

```



```

auto r33 = std::scan<int>("#*42*", "{:*^}"); // ERROR (invalid_scanned
auto r34 = std::scan<int>("#*42*", "#{:*^}"); // r34->value() == 42, r3
auto r35 = std::scan<int>("#*42*", "#{:#^}"); // ERROR (invalid_scanned

auto r36 = std::scan<int>("***42*", "{:*^3}"); // r36->value() == 42,
auto r37 = std::scan<int>("***42*", "{:*^.3}"); // ERROR (invalid_fill)

```

§ 4.3.3. Sign, #, and 0

std-format-spec:

... ~~sign~~_{opt} ~~#~~_{opt} ~~0~~_{opt} ...

~~sign: one of~~

~~+ - space~~

These flags would have no effect in `std::scan`, so they are disabled. Signs (both `+` and `-`), base prefixes, trailing decimal points, and leading zeroes are always allowed for arithmetic values. Disabling them would be a bad default for a higher-level facility like `std::scan`, so flags explicitly enabling them are not needed. Allowing them would just be misleading and lead to confusion about their behavior.

NOTE: This is incompatible with `std::format` format strings.

§ 4.3.4. Width and precision

width:

positive-integer

~~{arg-id}~~_{opt}

precision:

nonnegative-integer

~~{arg-id}~~_{opt}

The width and precision specifiers are valid for all argument types. Their meaning is virtually the same as with `std::format`: the width specifies the minimum field width, whereas the precision specifies the maximum. The scanned value itself, and any fill characters are counted as a part of said field width.

Either one of these can be specified to set either a minimum or a maximum, or both to provide a range of valid field widths.

Having a value shorter than the minimum field width is an error. Having a value longer than the maximum field width is not possible: reading will be cut short once the maximum field width is reached. If the value parsed up to that point is not a valid value, an error is provided.

EXAMPLE 4

```
// Minimum width of 2
auto r0 = std::scan<int>("123", "{:2}");
// r0->value() == 123, r0->range() == ""

// Maximum width of 2
auto r1 = std::scan<int>("123", "{:.2}");
// r1->value() == 12, r1->range() == "3"
```

For compatibility with `std::format`, the width and precision specifiers are in *field width units*, which is specified to be 1 per Unicode (extended) grapheme cluster, except some grapheme clusters are 2 ([format.string.std] ¶ 13):

For a sequence of characters in UTF-8, UTF-16, or UTF-32, an implementation should use as its field width the sum of the field widths of the first code point of each extended grapheme cluster. Extended grapheme clusters are defined by UAX #29 of the Unicode Standard. The following code points have a field width of 2:

- any code point with the East_Asian_Width="W" or East_Asian_Width="F" Derived Extracted Property as described by UAX #44 of the Unicode Standard
- U+4dc0 – U+4dff (Yijing Hexagram Symbols)
- U+1f300 – U+1f5ff (Miscellaneous Symbols and Pictographs)
- U+1f900 – U+1f9ff (Supplemental Symbols and Pictographs)

The field width of all other code points is 1.

For a sequence of characters in neither UTF-8, UTF-16, nor UTF-32, the field width is unspecified.

This essentially maps 1 field width unit = 1 user perceived character. It should be noted, that with this definition, grapheme clusters like emoji have a field width of 2. This behavior is present in

`std::format` today, but can potentially be surprising to users.

This meaning for both the width and precision specifiers are different from `scanf`, where the width means the number of code units to read. This is because the purpose of that specifier in `scanf` is to prevent buffer overflow. Because the current interface of the proposed `std::scan` doesn't allow reading into an user-defined buffer, this isn't a concern.

Specifying the width with another argument, like in `std::format`, is disallowed.

§ 4.3.5. Localized (L)

std-format-spec:
... L_{opt} ...

Enables scanning of values in locale-specific forms.

- For integer types, allows for digit group separator characters, equivalent to `numpunct::thousands_sep` of the used locale. If digit group separator characters are used, their grouping doesn't have to match `numpunct::grouping`.
- For floating-point types, the same as above. In addition, the locale-specific radix separator character is used, from `numpunct::decimal_point`.
- For `bool`, the textual representation uses the appropriate strings from `numpunct::truename` and `numpunct::falsename`.

§ 4.3.6. Type specifiers: strings

Type	Meaning
none, <code>s</code>	Copies from the input until a whitespace character is encountered.
<code>?</code>	Copies an escaped string from the input.
<code>c</code>	Copies from the input until the field width is exhausted. Does not skip preceding whitespace. Errors, if no field width is provided.

NOTE: The `s` specifier is consistent with `std::istream` and `std::string`:

```
std::string word;
std::istringstream{"Hello world"} >> word;
// word == "Hello"

auto r = std::scan<string>("Hello world", "{:s}");
// r->value() == "Hello"
```

NOTE: The `c` specifier is consistent with `scanf`, but is not supported for strings by `std::format`.

§ 4.3.7. Type specifiers: integers

Integer values are scanned as if by using `std::from_chars`, except:

- A positive `+` sign and a base prefix are always allowed to be present.
- Preceding whitespace is skipped.

Type	Meaning
<code>b, B</code>	<code>from_chars</code> with base 2. The base prefix is <code>0b</code> or <code>0B</code> .
<code>o</code>	<code>from_chars</code> with base 8. For non-zero values, the base prefix is <code>0</code> .
<code>x, X</code>	<code>from_chars</code> with base 16. The base prefix is <code>0x</code> or <code>0X</code> .
<code>d</code>	<code>from_chars</code> with base 10. No base prefix.
<code>u</code>	<code>from_chars</code> with base 10. No base prefix. No <code>-</code> sign allowed.
<code>i</code>	Detect base from a possible prefix, default to decimal.
<code>c</code>	Copies a character from the input.
none	Same as <code>d</code>

NOTE: The flags `u` and `i` are not supported by `std::format`. These flags are consistent with `scanf`.

NOTE: `[SCNLIB]` also supports the flag `0` for octal numbers, and `0o` and `0O` as possible octal number prefixes. These are currently not proposed.

§ 4.3.8. Type specifiers: `CharT`

Type	Meaning
none, <code>c</code>	Copies a character from the input.
<code>b</code> , <code>B</code> , <code>d</code> , <code>i</code> , <code>o</code> , <code>u</code> , <code>x</code> , <code>X</code>	Same as for integers.
<code>?</code>	Copies an escaped character from the input.

EXAMPLE 5

This is not encoding or Unicode-aware. Reading a `CharT` with the `c` type specifier will just read a single code unit of type `CharT`. This can lead to invalid encoding in the scanned values.

```
// As proposed:
// U+12345 is 0xF0 0x92 0x8D 0x85 in UTF-8
auto r = std::scan<char, std::string>("\u{12345}", "{}{}");
auto& [ch, str] = r->values();
// ch == '\xF0'
// str == "\x92\x8d\x85" (invalid utf-8)

// This is the same behavior as with iostreams today
```

§ 4.3.9. Type specifiers: `bool`

Type	Meaning
<code>s</code>	Allows for textual representation, i.e. <code>true</code> or <code>false</code>

<code>b</code> , <code>B</code> , <code>d</code> , <code>i</code> , <code>o</code> , <code>u</code> , <code>x</code> , <code>X</code>	Allows for integral representation, i.e. <code>0</code> or <code>1</code>
<code>none</code>	Allows for both textual and integral representation: i.e. <code>true</code> , <code>1</code> , <code>false</code> , or <code>0</code> .

§ 4.3.10. Type specifiers: floating-point types

Similar to integer types, floating-point values are scanned as if by using `std::from_chars`, except:

- A positive `+` sign is always allowed to be present.
- Preceding whitespace is skipped.

Type	Meaning
<code>a</code> , <code>A</code>	<code>from_chars</code> with <code>chars_format::hex</code> , with <code>0x/0X</code> -prefix allowed.
<code>e</code> , <code>E</code>	<code>from_chars</code> with <code>chars_format::scientific</code> .
<code>f</code> , <code>F</code>	<code>from_chars</code> with <code>chars_format::fixed</code> .
<code>g</code> , <code>G</code>	<code>from_chars</code> with <code>chars_format::general</code> .
<code>none</code>	<code>from_chars</code> with <code>chars_format::general</code> <code>chars_format::hex</code> , with <code>0x/0X</code> -prefix allowed.

§ 4.3.11. Type specifiers: pointers

`std::format` supports formatting pointers of type `void*` and `const void*`. For consistency's sake, `std::scan` also supports reading a `void*` or `const void*`. Unlike `std::format`, `std::nullptr_t` is not supported.

Type	Meaning
<code>none</code> , <code>p</code> , <code>P</code>	as if by reading a value of type <code>uintptr_t</code> with the <code>x</code> type specifier

§ 4.4. Ranges

We propose, that `std::scan` would take a range as its input. This range should satisfy the requirements of `std::ranges::forward_range` to enable look-ahead, which is necessary for parsing.

```
template <class Range, class CharT>
concept scannable-range =
    ranges::forward_range<Range> &&
    same_as<ranges::range_value_t<Range>, CharT> &&
    (same_as<CharT, char> || same_as<CharT, wchar_t>);
```

For a range to be a *scannable-range*, its character type (range *value_type*, code unit type) needs to also be correct, i.e. it needs to match the character type of the format string. Mixing and matching character types between the input range and the format string is not supported.

EXAMPLE 6

```
scan<int>("42", "{}");    // OK
scan<int>(L"42", L"{}");  // OK
scan<int>(L"42", "{}");   // Error: wchar_t\N is not a scannable-range<
```

It should be noted, that standard range facilities related to iostreams, namely `std::istreambuf_iterator`, model `input_iterator`. Thus, they can't be used with `std::scan`, and therefore, for example, `stdin`, can't be read directly using `std::scan`. The reference implementation deals with this by providing a range type, that wraps a `std::basic_istreambuf`, and provides a *forward_range*-compatible interface to it. At this point, this is deemed out of scope for this proposal.

As mentioned above, *forward_ranges* are needed to support proper lookahead and rollback. For example, when reading an `int` with the `i` format specifier (detect base from prefix), whether a character is part of the `int` can't be determined before reading past it.

EXAMPLE 7

```
// Hex value "0xf"
auto r1 = std::scan<int>("0xf", "{:i}");
// r1->value() == 0xf
// r1->range().empty() == true

// (Octal) value "0", with "xg" left over
auto r2 = std::scan<int>("0xg", "{:i}");
// r2->value() == 0
// r2->range() == "xg"

// Compare with sscanf:

int val{}, n{};
int r = std::sscanf("0xf", "%i%n", &val, &n);
// val == 0xf
// n == 3 -> remainder == ""
// r == 1 -> SUCCESS

r = std::sscanf("0xg", "%i%n", &val, &n);
// val == 0
// n == 2 -> remainder == "g"
// r == 1 -> SUCCESS
```

The same behavior can be observed with floating-point values, when using exponents: whether `1e+X` is parsed as a number, or as `1` with the rest left over, depends on whether `X` is a valid exponent. For user-defined types, arbitrarily-long lookback or rollback can be required.

§ 4.5. Argument passing, and return type of `scan`

`std::scan` is proposed to return the values it scans, wrapped in a `std::expected`.

```
auto result = std::scan<int>(input, "{}");
auto [i] = result->values();
// or (only a single scanned value):
auto i = result->value();
```

The rationale for this is as follows:

- With output parameters, it would be easy to accidentally use uninitialized values. With return values, the values can only be accessed when the operation is successful.
- Modern C++ API design principles favor return values over output parameters.

It should be noted, that not using output parameters removes a channel for user customization. For example, `[FMT]` uses `fmt::arg` to specify named arguments, and `fmt::format_as` for easy formatting of enumerators. The same isn't directly possible here, without customizing the type to be scanned itself.

The return type of `scan`, `scan_result`, contains a `subrange` over the unparsed input. This can be accessed with the member function `range()`. This is done with an exposition-only type alias, `borrowed-tail-subrange-t`, that is defined as follows:

```
template <typename R>
using borrowed_tail_subrange_t = std::conditional_t<
    ranges::borrowed_range<R>,
    ranges::subrange<ranges::iterator_t<R>, ranges::sentinel_t<R>>,
    ranges::dangling>;
```

Compare this with `borrowed_subrange_t`, which is defined as `ranges::subrange<ranges::iterator_t<R>, ranges::iterator_t<R>>`, when the range models `borrowed_range`. This kind of subrange is returned to avoid having to advance to the end of the range in order to return an iterator pointing to it: we can just return the sentinel we're given, instead.

In addition to a subrange, as pointed out above, the success side of the returned expected also contains a `tuple` of the scanned values. This tuple can be retrieved with the `values()` member function, or if there's only a single scanned value, also with `value()`.

§ 4.5.1. Design alternatives

As proposed, `std::scan` returns an `expected`, containing either an iterator and a tuple, or a `scan_error`.

An alternative could be returning a `tuple`, with a result object as its first (0th) element, and the parsed values occupying the rest. This would enable neat usage of structured bindings:

```
// NOT PROPOSED, design alternative
auto [r, i] = std::scan<int>("42", "{}");
```

However, there are two possible issues with this design:

1. It's easy to accidentally skip checking whether the operation succeeded, and access the scanned values regardless. This could be a potential security issue (even though the values would always be at least value-initialized, not default-initialized). Returning an `expected` forces checking for success.
2. The numbering of the elements in the returned tuple would be off-by-one compared to the indexing used in format strings:

```
auto r = std::scan<int>("42", "{0}");  
// std::get<0>(r) refers to the result object  
// std::get<1>(r) refers to {0}
```

For the same reason as enumerated in 2. above, the `scan_result` type as proposed doesn't follow the tuple protocol, so that structured bindings can't be used with it:

```
// NOT PROPOSED  
auto result = std::scan<int>("42", "{0}");  
// std::get<0>(*result) would refer to the iterator  
// std::get<1>(*result) would refer to {0}
```

§ 4.6. Error handling

Contrasting with `std::format`, this proposed library communicates errors with return values, instead of throwing exceptions. This is because error conditions are expected to be much more frequent when parsing user input, as opposed to text formatting. With the introduction of `std::expected`, error handling using return values is also more ergonomic than before, and it provides a vocabulary type we can use here, instead of designing something novel.

`std::scan_error` holds an enumerated error code value, and a message string. The message is used in the same way as the message in `std::exception`: it gives more details about the error, but its contents are unspecified.

```
// Not a specification, just exposition  
class scan_error {  
public:  
    enum code {  
        // Tried to read from an empty range,  

```

```

// or the input ended unexpectedly.
end_of_input,

// The format string was invalid:
// This will often be caught at compile time,
// except when using `std::runtime_format`.
invalid_format_string,

// A generic error, for when the input
// did not contain a valid representation
// for the type to be scanned.
invalid_scanned_value,

// Literal character specified in the format string
// was not found in the source.
invalid_literal,

// Too many fill characters scanned,
// field precision (maximum field width) exceeded.
invalid_fill,

// Scanned field width was shorter than
// what was specified as the minimum field width.
length_too_short,

// Value too large (higher than the maximum value)
value_positive_overflow,

// Value too small (lower than the minimum value)
value_negative_overflow,

// Value magnitude too small, sign +
// (between 0 and the smallest subnormal)
value_positive_underflow,

// Value magnitude too small, sign -
// (between 0 and the smallest subnormal)
value_negative_underflow
};

constexpr scan_error(enum code, const char*);

```

```
constexpr auto code() const noexcept -> enum code;
constexpr const char* msg() const;
};
```

NOTE: [SCNLIB] has an additional error code enumerator, `invalid_source_state`. It's currently used when the input is not a range, but something like a file or an `istream`. As these kinds of input are currently not supported with this proposal, this is not proposed.

NOTE: A previous revision of this proposal had fewer enumerators, with the overflow/underflow enumerators being one `value_out_of_range`, and `invalid_literal`, `invalid_fill`, and `length_too_short` being folded into `invalid_scanned_value`. The added granularity provided in this revision was found to be useful.

The reason why we propose adding the type `std::scan_error` instead of just using `std::errc` is, that we want to avoid losing information. The enumerators of `std::errc` are insufficient for this use, as evident by the table below: there are no clear one-to-one mappings between `enum scan_error::code` and `std::errc`, but `std::errc::invalid_argument` would need to cover a lot of cases. Also, `std::errc` has a lot of unnecessary error codes, and a

The `const char*` in `scan_error` is extremely useful for user code, for use in logging and debugging. Even with the `enum scan_error::code` enumerators, more information is often needed, to isolate any possible problem.

Possible mappings from `enum scan_error::code` to `std::errc` could be:

<code>enum scan_error::code</code>	<code>errc</code>
<code>scan_error::end_of_input</code>	<code>std::errc::invalid_argument</code>
<code>scan_error::invalid_format_string</code>	
<code>scan_error::invalid_scanned_value</code>	
<code>scan_error::invalid_literal</code>	
<code>scan_error::invalid_fill</code>	
<code>scan_error::length_too_short</code>	
<code>scan_error::value_positive_overflow</code>	<code>std::errc::result_out_of_range</code>

<code>enum scan_error::code</code>	<code>errc</code>
<code>scan_error::value_negative_overflow</code>	
<code>scan_error::value_positive_underflow</code>	
<code>scan_error::value_negative_underflow</code>	

NOTE: [\[SCNLIB\]](#) provides a member function, `scan_error::to_errc()`, that performs this mapping.

Currently, as proposed, the message contained in a `scan_error` is of type `const char*`. Additionally, the validity of this message is only guaranteed up until the next call to a scanning function. This allows for performant use of string literals, but also leaves the opportunity for the implementation to do interesting things, for example by using thread-local storage to construct a custom error message, without allocating or using a `std::string`. Using `std::string` here would needlessly bloat up the type, both in terms of its size and its performance.

[\[SCNLIB\]](#) currently only uses string literals for its error messages, except when a user-defined `scanner::parse` throws a `scan_format_string_error`, for which TLS is utilized. See [§ 4.9 Extensibility](#) below for more details.

§ 4.7. Binary footprint and type erasure

We propose using a type erasure technique to reduce the per-call binary code size. The scanning function that uses variadic templates can be implemented as a small inline wrapper around its non-variadic counterpart:

```
template<scannable-range<char> Range>
auto vscan(Range&& range, string_view fmt, scan_args args)
    -> expected<ranges::borrowed_tail_subrange_t<Range>, scan_error>;

template <typename... Args, scannable-range<char> SourceRange>
auto scan(SourceRange&& source, scan_format_string<Range, Args...> format)
    -> expected<
        scan_result<ranges::borrowed_tail_subrange_t<SourceRange>, Args...
        scan_error> {
    auto result = make_scan_result<Source, Args...>();
    fill_scan_result(result, vscan(std::forward<SourceRange>(range), format,
```

```

                                make_scan_args(result->values())));
    return result;
}

```

As shown in [P0645] this dramatically reduces binary code size, which will make `scan` comparable to `scanf` on this metric.

`make_scan_args` type erases the arguments that are to be scanned. This is similar to `std::make_format_args`, used with `std::format`.

`make_scan_result` returns a default-constructed `expected`, containing an empty subrange and a tuple of value-initialized arguments. This is the value that will be returned from `scan`. The values will be populated by `vscan`, which will be given a reference to these values through the type-erased `scan_args`. The subrange will be set by `fill_scan_result`, which is described below. This approach allows us to take advantage of NRVO, which will eliminate copies and moves of the scan argument tuple out of `scan` into the caller's scope.

`fill_scan_result` takes the return value of `vscan`, and either writes the leftover range indicated by it into `result`, or writes an error. It's essentially one-liner sugar for this:

```

void fill_scan_result(auto& result, auto&& vscan_result) {
    // skipping type checking
    if (vscan_result) {
        result->set_range(*vscan_result);
    } else {
        result = unexpected(vscan_result.error());
    }
}

```

NOTE: This implementation of `std::scan` is more complicated compared to `std::format`, which can be described as a one-liner calling `std::vformat`. This is because the arguments that are written to by `vscan` need to outlive the call to `vscan`, so that they can be safely returned from `scan`.

A previous revision of this proposal used a different approach to type erasure and the implementation of `scan`. In that approach, `scan_arg_store` would store both a `tuple` of scanning arguments, and an array of `basic_scan_args`, that erased these arguments. Then, after calling `vscan`, the return object would be constructed by moving the `tuple` into it.

This had comparatively very bad codegen and performance for non-trivially copyable types, as copying or moving them on return couldn't be elided. Compare this to the current approach, where we don't have an intermediary tuple, but construct the return object straight away, and write directly to it.

§ 4.8. Safety

`scanf` is arguably more unsafe than `printf` because `__attribute__((format(scanf, ...)))` ([ATTR]) implemented by GCC and Clang doesn't catch the whole class of buffer overflow bugs, e.g.

```
char s[10];
std::sscanf(input, "%s", s); // s may overflow.
```

Specifying the maximum length in the format string above solves the issue but is error-prone, especially since one has to account for the terminating null.

Unlike `scanf`, the proposed facility relies on variadic templates instead of the mechanism provided by `<cstdarg>`. The type information is captured automatically and passed to scanners, guaranteeing type safety and making many of the `scanf` specifiers redundant (see § 4.2 Format strings). Memory management is automatic to prevent buffer overflow errors.

§ 4.9. Extensibility

We propose an extension API for user-defined types similar to `std::formatter`, used with `std::format`. It separates format string processing and parsing, enabling compile-time format string checks, and allows extending the format specification language for user types. It enables scanning of user-defined types.

```
auto r = scan<tm>(input, "Date: {0:%Y-%m-%d}");
```

This is done by providing a specialization of `scanner` for `tm`:

```
template <>
struct scanner<tm> {
    template <class ParseContext>
    constexpr auto parse(ParseContext& ctx)
        -> typename ParseContext::iterator;
```

```
template <class ScanContext>
auto scan(tm& t, ScanContext& ctx) const
    -> expected<typename ScanContext::iterator, scan_error>;
};
```

The `scanner<tm>::parse` function parses the `format-spec` portion of the format string corresponding to the current argument, and `scanner<tm>::scan` parses the input range `ctx.range()` and stores the result in `t`.

An implementation of `scanner<T>::scan` can potentially use the istream extraction `operator>>` for user-defined type `T`, if available.

Error handling in `scanner::parse` differs from the other parts of this proposal. To facilitate better compile time error checking, `parse` doesn't return an `expected`. Instead, to report errors, it can throw an exception of type `std::scan_format_string_error`, which is an exception type derived from `std::runtime_error`.

Then, if `parse` is being executed at compile time, and it throws, it makes the program ill-formed (`throw` is not constant expression). This also makes the compiler error message easy to read, as it'll point right where the `throw` expression is, with the error description. If `parse` is executed at run time, the exception is caught in the library, and eventually returned from `std::scan` inside a `scan_error`, with the error code of `invalid_format_string`.

A previous revision of this paper proposed returning `expected<typename ParseContext::iterator, scan_error>` from `parse`. While consistent with `scan`, it had the issue of diminished quality of compiler error messages. Returning an `unexpected` value from `parse` was not a compile-time error onto itself, so the compile-time error only manifested from inside the library, where it no longer had access to the original context and error message. By `throwing`, the compiler can point literally to the very line of code that reported the error.

NOTE: [\[SCNLIB\]](#) supports an additional means of error reporting from `parse`. `basic_scan_parse_context` has a member function, `on_error(const char*)`, that's not `constexpr`. This is useful for customers who aren't using exceptions, but it's not proposed in this paper.

§ 4.10. Locales

As pointed out in [\[N4412\]](#):

There are a number of communications protocol frameworks in use that employ text-based representations of data, for example XML and JSON. The text is machine-generated and machine-read and should not depend on or consider the locales at either end.

To address this, `std::format` provided control over the use of locales. We propose doing the same for the current facility by performing locale-independent parsing by default and designating separate format specifiers for locale-specific ones. In particular, locale-specific behavior can be opted into by using the `L` format specifier, and supplying a `std::locale` object.

EXAMPLE 8

```
std::locale::global(std::locale::classic());

// {} uses no locale
// {:L} uses the global locale
auto r0 = std::scan<double, double>("1.23 4.56", "{} {:L}");
// r0->values(): (1.23, 4.56)

// {} uses no locale
// {:L} uses the supplied locale
auto r1 = std::scan<double, double>(std::locale{"fi_FI"}, "1.23 4,56", '
// r1->values(): (1.23, 4.56)
```

§ 4.11. Encoding

In a similar manner as with `std::format`, input given to `std::scan` is assumed to be in the (ordinary/wide) literal encoding.

If an error in encoding is encountered while reading a value of a string type (`basic_string`, `basic_string_view`), an `invalid_scanned_value` error is returned. For other types, the reading is stopped, as the parser can't parse a numeric value from something that isn't digits, indirectly causing an error.

EXAMPLE 9

```
// Invalid UTF-8
auto r = std::scan<std::string>("a\xc3 ", "{}");
// r == false
// r->error() == std::scan_error::invalid_scanned_value

auto r2 = std::scan<int>("1\xc3 ", "{}");
// r2 == true
// r2->value() == 1
// r2->range() == "\xc3 "
```

Reading raw bytes (not in the literal encoding) into a `string` isn't directly supported. This can be achieved either with simpler range algorithms already in the standard, or by using a custom type or scanner.

§ 4.12. Performance

The API allows efficient implementation that minimizes virtual function calls and dynamic memory allocations, and avoids unnecessary copies. In particular, since it doesn't need to guarantee the lifetime of the input across multiple function calls, `scan` can take `string_view` avoiding an extra string copy compared to `std::istream`. Since, in the default case, it also doesn't deal with locales, it can internally use something like `std::from_chars`.

We can also avoid unnecessary copies required by `scanf` when parsing strings, e.g.

```
auto r = std::scan<std::string_view, int>("answer = 42", "{} = {}");
```

Because the format strings are checked at compile time, while being aware of the exact types to scan, and the source range type, it's possible to check at compile time, whether scanning a `string_view` would dangle, or if it's possible at all (reading from a non-`contiguous_range`).

§ 4.13. Integration with chrono

The proposed facility can be integrated with `std::chrono::parse` ([P0355]) via the extension mechanism, similarly to the integration between chrono and text formatting proposed in [P1361]. This

will improve consistency between parsing and formatting, make parsing multiple objects easier, and allow avoiding dynamic memory allocations without resolving to the deprecated [stringstream](#).

Before:

```
std::istringstream is("start = 10:30");
std::string key;
char sep;
std::chrono::seconds time;
is >> key >> sep >> std::chrono::parse("%H:%M", time);
```

After:

```
auto result = std::scan<std::string, std::chrono::seconds>("start = 10:30",
const auto& [key, time] = result->values();
```

Note that the [scan](#) version additionally validates the separator.

Scanning of time points, clock values, and calendar values is implemented in [\[SCNLIB\]](#).

§ 4.14. Impact on existing code

The proposed API is defined in a new header and should have no impact on existing code.

§ 5. Existing work

[\[SCNLIB\]](#) is a C++ library that serves as the reference implementation of this proposal. Its interface and behavior follows the design described in this paper.

[\[FMT\]](#) has a prototype implementation of an earlier version of the proposal.

§ 6. Future extensions

To keep the scope of this paper somewhat manageable, we've chosen to only include functionality we consider fundamental. This leaves the design space open for future extensions and other proposals. However, we are not categorically against exploring this design space, if it is deemed critical for v1.

All of the possible future extensions described below are implemented in [\[SCNLIB\]](#).

§ 6.1. Integration with `stdio`

In the SG9 meeting in Kona (11/2023), it was polled, that:

SG9 feels that it essential for `std::scan` to be useable with `stdin` and `cin` (and the paper would be incomplete without this feature).

SF	F	N	A	SA
0	5	1	3	0

We've decided to follow the route of `std::format` + `std::print`, i.e. to not complicate and bloat this paper further by involving I/O. This is still an important avenue of future expansion, and the library proposed in this paper is designed and specified in such a way as to easily allow that expansion.

[\[SCNLIB\]](#) implements this by providing a function, `scn::input`, for interfacing with `stdin`, and by allowing passing in `FILE*`s as input to `scn::scan`, in addition to `scannable-ranges`.

§ 6.2. `scanf`-like `[character set]` matching

`scanf` supports the `[]` format specifier, which allows for matching for a set of accepted characters. Unfortunately, because some of the syntax for specifying that set is implementation-defined, the utility of this functionality is hampered. Properly specified, this could be useful.

EXAMPLE 10

```
auto r = scan<string>("abc123", "{:[a-zA-Z]}"); // r->value() == "abc",
// Compare with:
char buf[N];
sscanf("abc123", "%[a-zA-Z]", buf);

// ...

auto _ = scan<string>(..., "{:[^\n]}"); // match until newline
```

It should be noted, that while the syntax is quite similar, this is not a regular expression. This syntax is intentionally way more limited, as is meant for simple character matching.

This syntax is actually very useful when doing a little more complicated parsing, but it's still left out for the interest of scope.

[SCNLIB] implements this syntax, providing support for matching single characters/code points (`{:[abc]}`) and code point ranges (`{:[a-z]}`). Full regex matching is also supported with `{:/.../}`.

§ 6.3. Reading code points (or even grapheme clusters?)

`char32_t` nowadays the type denoting a Unicode code point. Reading individual code points, or even Unicode grapheme clusters, could be a useful feature. Currently, this proposal only supports reading of individual code units (`char` or `wchar_t`).

[SCNLIB] supports reading Unicode code points with `char32_t`.

§ 6.4. Reading strings and chars of different width

In C++, we have character types other than `char` and `wchar_t`, too: namely `char8_t`, `char16_t`, and `char32_t`. Currently, this proposal only supports reading strings with the same character type as the input range, and reading `wchar_t` characters from narrow `char`-oriented input ranges, as does `std::format`. `scanf` somewhat supports this with the `l`-flag (and the absence of one in `wscanf`). Providing support for reading differently-encoded strings could be useful.

EXAMPLE 11

```
// Currently supported:
auto r0 = scan<wchar_t>("abc", "{}");

// Not supported:
auto r1 = scan<char>(L"abc", L"{}");
auto r2 =
    scan<string, wstring, u8string, u16string, u32string>("abc def ghi jkl");
auto r3 =
    scan<string, wstring, u8string, u16string, u32string>(L"abc def ghi jkl");
```

§ 6.5. Scanning of ranges

Introduced in [P2286] for `std::format`, enabling the user to use `std::scan` to scan ranges, could be useful.

§ 6.6. Default values for scanned values

Currently, the values returned by `std::scan` are value-constructed, and assigned over if a value is read successfully. It may be useful to be able to provide an initial value different from a value-constructed one, for example, for preallocating a `string`, and possibly reusing it:

EXAMPLE 12

```
string str;
str.reserve(n);
auto r0 = scan<string>(..., "{", {std::move(str)});
// ...
r0->value().clear();
auto r1 = scan<string>(..., "{", {std::move(r0->value())});
```

This same facility could be also used for additional user customization, as pointed out in § 4.5 [Argument passing, and return type of scan](#).

§ 6.7. Assignment suppression / discarding values

`scanf` supports discarding scanned values with the `*` specifier in the format string. [SCNLIB] provides similar functionality through a special type, `scn::discard`:

EXAMPLE 13

```
int i;
scanf("%*d", &i);

auto r = scn::scan<scn::discard<int>>(..., "{");
auto [_] = r->values();
```

§ 7. Specification

This wording is still quite preliminary, and will require more work. Note the similarity and referencing to [format] in some parts.

This wording is done relative to [\[N4988\]](#).

§ 7.1. General

Add the header `<scan>` to the appropriate place in the "C++ library headers" table in [headers], respecting alphabetical order.

Add an entry for `__cpp_lib_scan` to the appropriate place in [version.syn], respecting alphabetical order. Set the value of the macro to the date of adoption of the paper.

```
#define __cpp_lib_scan 20XXXXL // also in <scan>
```

§ 7.2. Scanning [scan]

DRAFTING NOTE: This section ("Scanning" [scan]), is to be added to "General utilities library" [utilities]. The numbering of headings here is done relative to the rest of this document: they aren't intended to be section numbers in the standard. As of [\[N4988\]](#), the correct section number for "Scanning" [scan] would be 22.17.

§ 7.2.1. Header `<scan>` synopsis [scan.syn]

```
namespace std {  
    // [scan.fmt.string], class template basic_scan_format_string  
    template<class charT, class Range, class... Args>  
        struct basic_scan_format_string;  
  
    template<class Range, class... Args>  
        using scan_format_string =  
            basic_scan_format_string<char,  
                                     type_identity_t<Range>,  
                                     type_identity_t<Args>...>;  
}
```

```

template<class Range, class... Args>
    using wscan_format_string =
        basic_scan_format_string<wchar_t,
                                type_identity_t<Range>,
                                type_identity_t<Args>...>;

// [scan.error], class scan_error
class scan_error;

// [scan.format.error], class scan_format_string_error
class scan_format_string_error;

// [scan.result.result], class template scan_result
template<class Range, class... Args>
    class scan_result;

template<ranges::range R>
    using borrowed_tail_subrange_t =
        conditional_t<
            ranges::borrowed_range<R>,
            ranges::subrange<ranges::iterator_t<R>, ranges::sentinel_t<R>>,
            ranges::dangling>; // exposition on

template<class Range, class... Args>
    using scan_result_type = expected<
        scan_result<borrowed_tail_subrange_t<Range>, Args...>,
        scan_error>; // exposition on

// [scan.result], result types
template<class Source, class... Args>
    constexpr scan_result_type<Range, Args...>
        make_scan_result();

template<class Result, class Range>
    constexpr void fill_scan_result(expected<Result, scan_error>& out,
                                    expected<Range, scan_error>&& in);

template<class Range, class charT>
    concept scannable_range =
        ranges::forward_range<Range> &&
        same_as<ranges::range_value_t<Range>, charT> &&
        (same_as<charT, char> || same_as<charT, wchar_t>); // exposition on

```



```

// [scan.functions], scanning functions
template<class... Args, scannable-range<char> Range>
    scan-result-type<Range, Args...> scan(Range&& range,
                                           scan_format_string<Range, Args...> fmt,
                                           scan_args args);

template<class... Args, scannable-range<wchar_t> Range>
    scan-result-type<Range, Args...> scan(Range&& range,
                                           wscan_format_string<Range, Args...> fmt,
                                           scan_args args);

template<class... Args, scannable-range<char> Range>
    scan-result-type<Range, Args...> scan(const locale& loc, Range&& range,
                                           scan_format_string<Range, Args...> fmt,
                                           scan_args args);

template<class... Args, scannable-range<wchar_t> Range>
    scan-result-type<Range, Args...> scan(const locale& loc, Range&& range,
                                           wscan_format_string<Range, Args...> fmt,
                                           scan_args args);

template<class Range>
    using vscan-result-type = expected<
        borrowed-tail-subrange-t<Range>,
        scan_error>; // exposition only

template<scannable-range<char> Range>
    vscan-result-type<Range> vscan(Range&& range, string_view fmt, scan_args args);

template<scannable-range<wchar_t> Range>
    vscan-result-type<Range> vscan(Range&& range, wstring_view fmt, scan_args args);

template<scannable-range<char> Range>
    vscan-result-type<Range> vscan(const locale& loc,
                                   Range&& range,
                                   string_view fmt,
                                   scan_args args);

template<scannable-range<wchar_t> Range>
    vscan-result-type<Range> vscan(const locale& loc,
                                   Range&& range,
                                   wstring_view fmt,
                                   wscan_args args);

// [scan.context], class template basic_scan_context
template<class Range, class charT> class basic_scan_context;
using scan_context = basic_scan_context<unspecified, char>;

```

```

using wscan_context = basic_scan_context<unspecified, wchar_t>;

// [scan.scanner], class template scanner
template<class T, class charT = char>
    struct scanner;

// [scan.scannable], concept scannable
template<class T, class charT>
    concept scannable = see below;

// [scan.parse.ctx], class template basic_scan_parse_context
template<class charT>
    class basic_scan_parse_context;

using scan_parse_context = basic_scan_parse_context<char>;
using wscan_parse_context = basic_scan_parse_context<wchar_t>;

// [scan.args], class template basic_scan_args
template<class Context> class basic_scan_args;
using scan_args = basic_scan_args<scan_context>;
using wscan_args = basic_scan_args<wscan_context>;

// [scan.arg], class template basic_scan_arg
template<class Context>
    class basic_scan_arg;

// [scan.arg.store], class template scan-arg-store
template<class Context, class... Args>
    class scan-arg-store; // exposition only

template<class Context = scan_context, class... Args>
    constexpr scan-arg-store<Context, Args...>
        make_scan_args(std::tuple<Args...>& args);

template<class... Args>
    constexpr scan-arg-store<wscan_context, Args...>
        make_wscan_args(std::tuple<Args...>& args);
}

```

§ 7.2.2. Format string [scan.string]

§ 7.2.2.1. General [scan.string.general]

A *format string* for arguments `args` is a (possibly empty) sequence of *replacement fields*, *escape sequences*, *whitespace characters*, and characters other than `{` and `}`. Each character that is not part of a replacement field or an escape sequence, and is not whitespace character, is matched with a character in the input. An escape sequence is one of `{{` or `}}`. It is matched with `{` or `}`, respectively, in the input. For a sequence of characters in UTF-8, UTF-16, or UTF-32, any code point with the `Pattern_White_Space` property as described by UAX #31 of the Unicode standard is considered to be a whitespace character. For a sequence of characters in neither UTF-8, UTF-16, or UTF-32, the set of characters considered to be whitespace characters is unspecified. The syntax of replacement fields is as follows:

scan-replacement-field:

`{ arg-idopt scan-format-specifieropt }`

arg-id:

`0`

positive-integer

positive-integer:

nonzero-digit

positive-integer digit

nonnegative-integer:

digit

nonnegative-integer digit

nonzero-digit: one of

`1 2 3 4 5 6 7 8 9`

digit: one of

`0 1 2 3 4 5 6 7 8 9`

scan-format-specifier:

`:` *scan-format-spec*

scan-format-spec:

as specified by the `scanner` specialization for the argument type; cannot start with `}`

WORDING NOTE: [format.string.general] defines *replacement-field*, *arg-id*, *positive-integer*, *nonnegative-integer*, *nonzero-digit*, *digit*, *format-specifier*, and *format-spec* in the syntax for replacement fields. Our definitions are identical to these, except we define *scan-replacement-field*, *scan-format-specifier*, and *scan-format-spec* instead, and in *scan-format-spec*, we refer to `scanner` specializations instead of `formatter` specializations.

The *arg-id* field specifies the index of the argument in `args` whose value is to be scanned from the input instead of the replacement field. If there is no argument with the index *arg-id* in `args`, the string is not a format string for `args`. The optional *scan-format-specifier* field explicitly specifies a format for the scanned value.

[Example 1:

```
auto r = scan<int>("8-{}", "{}-{}"); // value of r->value() is 8
— end example]
```

If all *arg-ids* in a format string are omitted, argument indices 0, 1, 2, ... will automatically be used in that order. If some *arg-ids* are omitted and some are present, the string is not a format string. If there is any argument in *args* that doesn't have a corresponding replacement field, or if there are multiple replacement fields corresponding to an argument in *args*, the string is not a format string for *args*.

[Note 1: A format string cannot contain a mixture of automatic and manual indexing. Every argument to be scanned must have one and exactly one corresponding replacement field in the format string. — end note]

WORDING NOTE: This is stricter than what's required in [format.string.general]. We have the additional requirements of having to mention every argument in the format string, and not allowing duplication of arguments in the format string.

The *scan-format-spec* field contains *format specifications* that define how the value should be scanned. Each type can define its own interpretation of the *scan-format-spec* field. If *scan-format-spec* does not conform to the format specifications for the argument type referred to by *arg-id*, the string is not a format string for `args`.

[Example 2:

- For arithmetic, pointer, and string types the *scan-format-spec* is interpreted as a *std-scan-format-spec* as described in [scan.string.std].
- For user defined `scanner` specializations, the behavior of the `parse` member function determines how the *scan-format-spec* is interpreted.

— end example]

§ 7.2.2.2. Standard format specifiers [scan.string.std]

Each `scanner` specialization described in [scan.scanner.spec] for fundamental and string types interprets *scan-format-spec* and a *std-scan-format-spec*.

[*Note 1*: The format specification can be used to specify such details as minimum field width, alignment, and padding. Some of the formatting options are only supported for arithmetic types. — *end note*]

The syntax of format specifications is as follows:

std-scan-format-spec:
*fill-and-align*_{opt} *scan-width*_{opt} *scan-precision*_{opt} *L*_{opt} *scan-type*_{opt}
fill-and-align:
*fill*_{opt} *align*
fill:
any character other than { or }
align: one of
< > ^
scan-width:
positive-integer
scan-precision:
. *nonnegative-integer*
scan-type: one of
a A b B c d e E f F g G i o p P s u x X ?

Field widths are specified in field width units (see [format.string.std]).

The *fill character* is the character denoted by the *fill* option or, if the *fill* option is absent, the space character. For a format specification in UTF-8, UTF-16, or UTF-32, the fill character corresponds to a single Unicode scalar value. Fill characters are always assumed to have a field width of one.

[*Note 2*: The presence of a *fill* option is signaled by the character following it, which must be one of the alignment options. If the second character of *std-scan-format-spec* is not a valid alignment option, then it is assumed that the *fill* and *align* options are both absent. — *end note*]

The *align* option applies to all argument types. The meaning of the various alignment options is as specified in [tab:scan.align].

Meaning of *align* options [tab:scan.align]

Option	Meaning
<	Skips fill characters after the scanned value, until either a non-fill character is encountered, or the maximum field width is reached. If no <i>align</i> option is specified, but

Option	Meaning
	a <i>scan-width</i> or <i>scan-precision</i> is, this is the option used for non-arithmetic non-pointer types, <code>charT</code> , and <code>bool</code> , unless an integer presentation type is specified.
>	Skips fill characters before the scanned value, until either a non-fill character is encountered, or the maximum field width is reached. If the maximum field width is reached by only reading fill characters, an error with the code <code>scan_error::invalid_fill</code> is returned; If no <i>align</i> option is specified, but a <i>scan-width</i> or <i>scan-precision</i> is, this is the option used for arithmetic types other than <code>charT</code> and <code>bool</code> , pointer types, or when any integer presentation type is specified.
^	Skips fill characters both before and after the scanned value, until either a non-fill character is encountered, or the maximum field width is reached. If the maximum field width is reached by only reading fill characters, an error with the code <code>scan_error::invalid_fill</code> is returned; [Note 3: The number of fill characters doesn't have to be equal both before and after the value. — <i>end note</i>]

The *scan-width* option specifies the minimum field width. If the *scan-width* option is absent, the minimum field width is 0. Otherwise, the value of the *positive-integer* is interpreted as a decimal integer and used as the value of the option. If the number of characters consumed for scanning a value, including the value itself and fill characters used for alignment, but excluding possibly skipped preceding whitespace is less than the minimum field width, an error with the code `scan_error::length_too_short` is returned.

For the purposes of width computation, a string is assumed to be in a locale-independent, implementation-defined encoding.

WORDING NOTE: In [format.string.std], we additionally say

Implementations should use either UTF-8, UTF-16, or UTF-32, on platforms capable of displaying Unicode text in a terminal.

It's unclear if we can and/or should place a similar kind of normative recommendation here.

For a sequence of characters in UTF-8, UTF-16, or UTF-32, the algorithm for calculating field width is described in [format.string.std]. For a sequence of characters in neither UTF-8, UTF-16, or UTF-32, the field width is unspecified.

The *scan-precision* option specifies the maximum field width. If a maximum field width is specified, it's the maximum number of characters read from the source range for any given scanning argument, including the value itself and any fill characters used for alignment, but excluding any possibly discarded preceding whitespace. Reaching the maximum field width is not an error.

When the `L` option is used, the form used for scanning is called the *locale-specific form*. The `L` option is only valid for arithmetic types, and its effect depends upon the type.

- For integral types, the locale-specific form causes digit group separator characters to be accepted. These digit group separator characters are ignored in parsing, and their form is determined by the context's locale.
- For floating-point types, the locale-specific form causes the radix separator character and digit group separator characters to be accepted, as determined by the context's locale.
- For the textual representation of `bool`, the locale-specific form causes the accepted values to be determined as if by `numprint::truename` and `numprint::falsename` of the context's locale.

The *scan-type* determines how the data should be scanned. Unless otherwise specified, before scanning a value, all whitespace characters are read and discarded from the input, until encountering a character that is not a whitespace character.

If the value to be scanned is of type `basic_string_view<charT>`, and `ranges::contiguous_range<R> && ranges::borrowed_range<R>` is `false` for a source range of type `R`, the string is not a format string for `args`, when using `R` as the type of the source range.

The available string presentation types are specified in `[tab:scan.type.string]`.

Meaning of *scan-type* options for strings `[tab:scan.type.string]`

Type	Meaning
none, <code>s</code>	Copies characters from the input until a whitespace character is encountered.
<code>c</code>	Copies characters from the input until the maximum field width is reached. Preceding whitespace is not skipped. If no value is given for the <i>scan-precision</i> option, the string is not a format string for <code>args</code> .
<code>?</code>	Copies the escaped string (<code>[format.string.escaped]</code>) from the input.

The meaning of some non-string presentation types is defined in terms of a call to `from_chars`. In such cases, let `[first, last)` be a contiguous range of characters sourced from the input and `value` be the scanning argument value. Scanning is done as if by first copying characters from the input into `[first, last)` until the first character invalid for the presentation type is found, after which `from_chars` is called. If `[first, last)` is an empty range, an error with the code `invalid_scanned_value` is returned.

[Note 4: Additional padding and adjustments are performed prior to calling `from_chars` as specified by the format specifiers. — *end note*]

Integral types other than `bool` and `charT` are scanned as if by using an infinite precision integral type. If its value cannot be represented in the integral type to be scanned, an error with either the code `value_positive_overflow` is returned if the value was positive, and `value_negative_overflow` if the value was negative. If the presentation type allows it, integral types other than `bool` and `charT` can have a *base prefix*. This is not copied into range `[first, last)`.

The available integer presentation types for integral types other than `bool` and `charT` are specified in `[tab:scan.type.int]`.

[Example 1:

```
auto r0 = scan<int>("42", "{}"); // Value of `r0->value()` is `42`

auto r1 = scan<int, int, int>("42 42 42", "{:d} {:o} {:x}");
// Values of `r1->values()` are `42`, `042`, and `0x42`

auto r2 = scan<int>("1,234", "{:L}");
// Value of `r2->value()` can be `1234` (depending on the locale)
```

— *end example*]

Meaning of *scan-type* options for integer types `[tab:scan.type.int]`

Type	Meaning
b, B	<code>from_chars(first, last, value, 2)</code> ; the allowed base prefixes are <code>0b</code> and <code>0B</code> .
c	Copies a value of type <code>charT</code> from the input. Preceding whitespace is not skipped.
d	<code>from_chars(first, last, value, 10)</code> .

Type	Meaning
<code>i</code>	<code>from_chars(first, last, value, base)</code> ; the value of <code>base</code> is determined by the base prefix: - if the base prefix is <code>0b</code> or <code>0B</code>, the value of <code>base</code> is 2, - if the base prefix is <code>0x</code> or <code>0X</code>, the value of <code>base</code> is 16, - if the base prefix is <code>0</code>, the value of <code>base</code> is 8, - otherwise, the value of <code>base</code> is 10.
<code>o</code>	<code>from_chars(first, last, value, 8)</code> ; the allowed base prefix is <code>0</code> .
<code>u</code>	The same as <code>i</code> , except if the scanned value would be negative, an error with the code <code>invalid_scanned_value</code> is returned.
<code>x, X</code>	<code>from_chars(first, last, value, 16)</code> ; the allowed base prefixes are <code>0x</code> and <code>0X</code> .
<code>none</code>	The same as <code>d</code> .

The available `charT` presentation types are specified in `[tab:scan.type.char]`.

Meaning of *scan-type* options for `charT` `[tab:scan.type.char]`

Type	Meaning
<code>none, c</code>	Copies a value of type <code>charT</code> from the input. Preceding whitespace is not skipped.
<code>b, B, d, i, o, u, x, X</code>	As if by scanning an integer as specified in <code>[tab:scan.type.int]</code> . If the scanned value is negative, an error with the code <code>value_negative_underflow</code> is returned. If the scanned value cannot be represented in <code>charT</code> , an error with the code <code>value_positive_underflow</code> is returned.
<code>?</code>	Copies the escaped character (<code>[format.string.escaped]</code>) from the input. Preceding whitespace is not skipped.

The available `bool` presentation types are specified in `[tab:scan.type.bool]`.

Meaning of *scan-type* options for `bool` `[tab:scan.type.bool]`

Type	Meaning
<code>s</code>	Copies the textual representation, either <code>true</code> or <code>false</code> , from the input.
<code>b, B, d, i, o, u, x, X</code>	Copies the integral representation, either <code>0</code> or <code>1</code> , from the input.
<code>none</code>	Copies one of <code>true</code> , <code>false</code> , <code>0</code> , or <code>1</code> from the input.

Values of a floating-point type `F` are scanned as if by copying characters from the input into a contiguous range represented by `[first, last)`. Let `sign-value` represent the sign of the value.

- If the first non-whitespace character is `+`, `sign-value` is `+1.0`, and this character is discarded,
- if the first non-whitespace character is `-`, `sign-value` is `-1.0`, and this character is discarded,
- otherwise, `sign-value` is `+1.0`.

If the characters following the sign are `"inf"` or `"infinite"` (case insensitive), the scanning is stopped, and `copysign(numeric_limits<F>::infinity(), static_cast<F>(sign-value))` is scanned. If the characters following the sign are `"nan"` or `"nan(pattern)"`, where `pattern` is a sequence of alphanumeric characters and underscores (case insensitive), the scanning is stopped, and `copysign(numeric_limits<F>::quiet_nan(), static_cast<F>(sign-value))` is scanned. Otherwise, scanning is done as specified by the floating-point presentation type.

If the absolute value of the scanned value is larger than what can be represented by `F`, a `scan_error` with the following code is returned:

- `scan_error::value_positive_overflow` if `signbit(sign-value)` is `false`,
- otherwise `scan_error::value_negative_overflow`.

If the absolute value of the scanned value is between zero and the smallest denormal value of `F`, a `scan_error` with the following code is returned:

- `scan_error::value_positive_underflow` if `signbit(sign-value)` is `false`,
- otherwise `scan_error::value_negative_underflow`.

[Note 5: NaN payload is discarded. Scanning a literal `"infinity"` is not an overflow error. — *end note*]

The available floating-point presentation types and their meanings are specified in `[tab:scan.type.float]`.

WORDING NOTE: This wording needs some serious work.

Meaning of *scan-type* options for floating-point types [tab:scan.type.float]

Type	Meaning
a, A	<code>from_chars(first, last, value, chars_format::hex)</code> followed by <code>copysign(value, static_cast<F>(sign-value))</code> , except a prefix <code>"0x"</code> or <code>"0X"</code> is allowed and discarded.
e, E	<code>from_chars(first, last, value, chars_format::scientific)</code> followed by <code>copysign(value, static_cast<F>(sign-value))</code> .
f, F	<code>from_chars(first, last, value, chars_format::fixed)</code> followed by <code>copysign(value, static_cast<F>(sign-value))</code> .
g, G	<code>from_chars(first, last, value, chars_format::general)</code> followed by <code>copysign(value, static_cast<F>(sign-value))</code> .
none	• If <code>[first, last)</code> starts with <code>"0x"</code> or <code>"0X"</code>, equivalent to <code>a</code>, • otherwise, equivalent to <code>g</code>.

The available pointer presentation types are specified in [tab:scan.type.ptr].

Meaning of *scan-type* options for pointer types [tab:scan.type.ptr]

Type	Meaning
none, p, P	If <code>uintptr_t</code> is defined, equivalent to scanning a value of type <code>uintptr_t</code> with the <code>x</code> <i>scan-type</i> , followed by a <code>reinterpret_cast</code> to <code>void*</code> or <code>const void*</code> ; otherwise, implementation-defined. [Note 6: No special null-value, apart from <code>0</code> and <code>0x0</code> is supported — end note]

§ 7.2.3. Error reporting [scan.err]

Scanning functions report errors using `expected<T, scan_error>` ([expected]).

Exceptions of a type publicly derived from `scan_format_string_error` thrown from the `parse` member function of a user defined specialization of `scanner` are caught by the library, and returned from a scanning function as a `scan_error` with a code of `scan_error::invalid_format_string`, and an unspecified message.

Recommended practice: Implementations should capture the message of the thrown exception, and preserve it in the returned `scan_error`.

[Note 1: `scan_error` contains a message of type `const char*`, and exceptions contain a message of type `std::string`, so propagating the message in a lifetime- and thread-safe manner is not possible without using thread-local storage or a side-channel. Use of TLS is possible because of the validity guarantees of `scan_error`. — end note]

All other exceptions thrown by iterators and user defined specializations of `scanner` are propagated. Failure to allocate storage is reported by throwing an exception as described in [res.on.exception.handling].

§ 7.2.3.1. Class `scan_error` [`scan.error`]

```
namespace std {
    class scan_error {
        enum code code_;          // exposition only
        const char* message_;     // exposition only

    public:
        enum code {
            end_of_input,
            invalid_format_string,
            invalid_scanned_value,
            invalid_literal,
            invalid_fill,
            length_too_short,
            value_positive_overflow,
            value_negative_overflow,
            value_positive_underflow,
            value_negative_underflow
        };

        constexpr scan_error() noexcept;
        constexpr scan_error(enum code error_code, const char* message);
    };
}
```

```
constexpr auto code() const noexcept -> enum code { return code_; }
constexpr const char* msg() const;
};
}
```

The class `scan_error` defines the type of objects used to represent errors returned from the scanning library. It stores an error code, and a human-readable descriptive message.

```
constexpr scan_error(code_type error_code, const char* message);
```

Preconditions: `message` is either a null pointer, or points to a NTCTS ([defs.ntcts]).

Postconditions: `code() == error_code && strcmp(message, msg()) == 0`.

```
constexpr const char* msg() const;
```

Preconditions: No other scanning function has been called since the one that returned `*this`.

Returns: `message_`.

§ 7.2.3.2. Class `scan_format_string_error` [scan.format.error]

```
namespace std {
    class scan_format_string_error : public runtime_error {
    public:
        explicit scan_format_string_error(const string& what_arg);
        explicit scan_format_string_error(const char* what_arg);
    };
}
```

The class `scan_format_string_error` defines the type of objects thrown as exceptions to report errors in parsing format strings in the scanning library.

```
scan_format_string_error(const string& what_arg);
```

Postconditions: `strcmp(what(), what_arg.c_str()) == 0`.

```
scan_format_string_error(const char* what_arg);
```

Postconditions: `strcmp(what(), what_arg) == 0`.

§ 7.2.4. Result types [scan.result]

```
template<class Source, class... Args>
constexpr scan_result_type<Range, Args...> make_scan_result();
```

Effects: Equivalent to: `return scan_result_type<Range, Args...>();`

```
template<class Source, class... Args>
constexpr void fill_scan_result(expected<Result, scan_error>& out,
                               expected<Range, scan_error>&& in);
```

Constraints:

- `Result` is a specialization of `scan_result`, and
- `std::is_same_v<typename Result::range_type, Range>` is true.

Effects:

- If `in.has_value()` is false, assigns `unexpected(std::move(in.error()))` to `out`,
- if `std::is_same_v<typename Result::range_type, ranges::dangling>` is false, assigns `std::move(*in)` to `out.range_`,
- otherwise, does nothing.

§ 7.2.4.1. Class template `scan_result` [scan.result.result]

```
namespace std {
    template<class Range, class... Args>
    class scan_result {
        using tuple_type = tuple<Args...>
```

```

range_type range_; // exposition only
tuple<Args...> values_; // exposition only

inline constexpr bool is-dangling =
    is_same_v<Range, ranges::dangling>; // exposition only

public:
    using range_type = Range;
    using iterator = see below;
    using sentinel = see below;

    constexpr scan_result();

    constexpr scan_result(const scan_result&) = default;
    constexpr scan_result(scan_result&&) = default;

    constexpr scan_result(Range r, tuple<Args...>&& values);

    template<class OtherR, class... OtherArgs>
        constexpr explicit(see below) scan_result(OtherR&& r, tuple&& values);

    template<class OtherR, class... OtherArgs>
        constexpr explicit(see below) scan_result(const scan_result<OtherR, OtherArgs>&);

    template<class OtherR, class... OtherArgs>
        constexpr explicit(see below) scan_result(scan_result<OtherR, OtherArgs>&&);

    constexpr scan_result& operator=(const scan_result&) = default;
    constexpr scan_result& operator=(scan_result&&) noexcept(see below) = default;

    template<class OtherR, class... OtherArgs>
        constexpr scan_result& operator=(const scan_result<OtherR, OtherArgs>&) = default;

    template<class OtherR, class... OtherArgs>
        constexpr scan_result& operator=(scan_result<OtherR, OtherArgs...>&&) = default;

    constexpr range_type range() const { return range_; }

    constexpr iterator begin() const;
    constexpr sentinel end() const;

    template<class Self>
        constexpr auto&& values(this Self&&);

```

```

template<class Self>
    constexpr auto&& value(this Self&&);
};
}

```

An instance of `scan_result` holds the scanned values and the remainder of the source range not used for scanning.

If a program declares an explicit or partial specialization of `scan_result`, the program is ill-formed, no diagnostic required.

`Range` shall either be a specialization of `ranges::subrange`, or `ranges::dangling`.

`conjunction_v<is_default_constructible<Args>...>` shall be `true`.

`conjunction_v<is_destructible<Args>...>` shall be `true`.

If `conjunction_v<is_trivially_destructible<Range>, is_trivially_destructible<Args>...>` is `true` then the destructor of `scan_result` is trivial.

```

using iterator = see below;
using sentinel = see below;

```

The type `iterator` is:

- If `is-dangling` is false, `ranges::iterator_t<Range>`,
- otherwise, `ranges::dangling`;

The type `sentinel` is:

- If `is-dangling` is false, `ranges::sentinel_t<Range>`,
- otherwise, `ranges::dangling`;

```
constexpr scan_result();
```

Effects: Value-initializes `range_` and `values_`.

```
constexpr scan_result(const scan_result& rhs) = default;
```


Mandates:

- `is_copy_constructible_v<Range>` is `true`, and
- `is_copy_constructible_v<tuple_type>` is `true`.

Effects: Direct-non-list-initializes `range_` with `rhs.range_`, and `values_` with `rhs.values_`.

```
constexpr scan_result(scan_result&& rhs) = default;
```

Constraints:

- `is_move_constructible_v<Range>` is `true`, and
- `is_move_constructible_v<tuple_type>` is `true`.

Effects: Direct-non-list-initializes `range_` with `std::move(rhs.range_)`, and `values_` with `std::move(rhs.values_)`.

```
constexpr scan_result(Range r, tuple<Args...>&& values);
```

Effects: Direct-non-list-initializes `range_` with `r`, and `values_` with `std::move(values)`.

```
template<class OtherR, class... OtherArgs>  
constexpr explicit(see below) scan_result(OtherR&& r, tuple&& values);
```

Constraints:

- `is_constructible_v<Range, OtherR>` is `true`, and
- `is_constructible_v<tuple<Args...>, tuple<OtherArgs...>>` is `true`.

Effects: Direct-non-list-initializes `range_` with `std::forward<OtherR>(r)`, and `values_` with `std::move(values)`.

Remarks: The expression inside `explicit` is equivalent to: `is_convertible_v<OtherR, Range> && is_convertible_v<tuple<OtherArgs>, tuple<Args>>`.

```
template<class OtherR, class... OtherArgs>  
constexpr explicit(see below) scan_result(const scan_result<OtherR, OtherArgs>&& rhs);
```

Constraints:

- `is_constructible_v<Range, const OtherR&>` is `true`, and
- `is_constructible_v<tuple<Args...>, const tuple<OtherArgs...>&>` is `true`.

Effects: Direct-non-list-initializes `range_` with `other.range_`, and `values_` with `other.values_`.

Remarks: The expression inside `explicit` is equivalent to: `is_convertible_v<const OtherR&, Range> && is_convertible_v<const tuple<OtherArgs>&, tuple<Args>>`.

```
template<class OtherR, class... OtherArgs>
constexpr explicit(see below) scan_result(scan_result<OtherR, OtherArgs>
```

Constraints:

- `is_constructible_v<Range, OtherR>` is `true`, and
- `is_constructible_v<tuple<Args...>, tuple<OtherArgs...>>` is `true`.

Effects: Direct-non-list-initializes `range_` with `std::move(other.range_)`, and `values_` with `std::move(other.values_)`.

Remarks: The expression inside `explicit` is equivalent to: `is_convertible_v<OtherR, Range> && is_convertible_v<tuple<OtherArgs>, tuple<Args>>`.

```
constexpr scan_result& operator=(const scan_result& rhs) = default;
```

Effects: Assigns `rhs.range_` to `range_`, and `rhs.values_` to `values_`.

Returns: `*this`.

Remarks: This operator is defined as deleted unless `is_copy_assignable_v<tuple<Args...>>` is `true`.

```
constexpr scan_result& operator=(scan_result&& rhs) noexcept(see below) = (
```

Constraints: `is_move_assignable_v<tuple<Args...>>` is `true`.

Effects: Assigns `std::move(rhs.range_)` to `range_`, and `std::move(rhs.values_)` to `values_`.

Returns: `*this`.

Remarks: The exception specification is equivalent to `is_nothrow_move_assignable_v<tuple<Args...>>`.

```
template<class OtherR, class... OtherArgs>
constexpr scan_result& operator=(const scan_result<OtherR, OtherArgs...>&
```

Constraints:

- `is_assignable_v<Range&, const OtherR&>` is true, and
- `is_assignable_v<tuple<Args...>&, const tuple<OtherArgs...>&>` is true

Effects: Assigns `rhs.range_` to `range_`, and `rhs.values_` to `values_`.

Returns: `*this`.

```
template<class OtherR, class... OtherArgs>
constexpr scan_result& operator=(scan_result<OtherR, OtherArgs...>&& rhs
```

Constraints:

- `is_assignable_v<Range&, OtherR>` is true, and
- `is_assignable_v<tuple<Args...>&, tuple<OtherArgs...>>` is true

Effects: Assigns `std::move(rhs.range_)` to `range_`, and `std::move(rhs.values_)` to `values_`.

Returns: `*this`.

```
constexpr iterator begin() const;
```

Returns:

- If `is_dangling` is false, `ranges::begin(range_)`,

- otherwise, a value-initialized object of type `ranges::dangling`.

```
constexpr sentinel end() const;
```

Returns:

- If `is-dangling` is false, `ranges::end(range_)`,
- otherwise, a value-initialized object of type `ranges::dangling`.

```
template<class Self>
constexpr auto&& values(this Self&& self);
```

Returns: `std::forward<Self>(self).values_`.

```
template<class Self>
constexpr auto&& value(this Self&& self);
```

Constraints: `sizeof...(Args)` is 1.

Returns: `get<0>(std::forward<Self>(self).values_)`.

§ 7.2.5. Class template `basic_scan_format_string` [scan.fmt.string]

```
namespace std {
    template<class charT, class Range, class... Args>
    struct basic_scan_format_string {
    private:
        basic_string_view<charT> str; // exposition only

    public:
        template<class T> consteval basic_scan_format_string(const T& s);
        basic_scan_format_string(runtime-format-string<charT> s) noexcept : str{ s }

        constexpr basic_string_view<charT> get() const noexcept { return str; }
    };
}
```

```
template<class T> consteval basic_scan_format_string(const T& s);
```

Constraints: `const T&` models `convertible_to<basic_string_view<charT>>`.

Effects: Direct-non-list-initializes `str` with `s`.

Remarks: A call to this function is not a core constant expression ([`expr.const`]) unless there exist `args` of types `Args` such that `str` is a format string for `args`.

§ 7.2.6. Scanning functions [`scan.functions`]

```
template<class... Args, scannable-range<char> Range>
    scan-result-type<Range, Args...> scan(Range&& range,
                                           scan_format_string<Range, Args...>
```

Effects: Let `result` be a value-initialized object of type `scan-result-type<Range, Args...>`. Creates an object `r` and initializes it with `vscan(std::forward<Range>(range), fmt.str, make_scan_args(result->values()))`.

- If `r.has_value()` is true, sets `result.range_` to `*r`,
- otherwise, assigns `unexpected(r.error())` to `result`.

Returns: `result`.

```
template<class... Args, scannable-range<wchar_t> Range>
    scan-result-type<Range, Args...> scan(Range&& range,
                                           wscan_format_string<Range, Args...>
```

Effects: Let `result` be a value-initialized object of type `scan-result-type<Range, Args...>`. Creates an object `r` and initializes it with `vscan(std::forward<Range>(range), fmt.str, make_wscan_args(result->values()))`.

- If `r.has_value()` is true, sets `result.range_` to `*r`,
- otherwise, assigns `unexpected(r.error())` to `result`.

Returns: `result`.

```
template<class... Args, scannable-range<char> Range>
    scan-result-type<Range, Args...> scan(const locale& loc, Range&& range,
                                           scan_format_string<Range, Args...>
```

Effects: Let `result` be a value-initialized object of type `scan-result-type<Range, Args...>`. Creates an object `r` and initializes it with `vscan(loc, std::forward<Range>(range), fmt.str, make_scan_args(result->values()))`.

- If `r.has_value()` is true, sets `result.range_` to `*r`,
- otherwise, assigns `unexpected(r.error())` to `result`.

Returns: `result`.

```
template <class... Args, scannable-range<wchar_t> Range>
    scan-result-type<Range, Args...> scan(const locale& loc, Range&& range,
                                           wscan_format_string<Range, Args...>
```

Effects: Let `result` be a value-initialized object of type `scan-result-type<Range, Args...>`. Creates an object `r` and initializes it with `vscan(loc, std::forward<Range>(range), fmt.str, make_wscan_args(result->values()))`.

- If `r.has_value()` is true, sets `result.range_` to `*r`,
- otherwise, assigns `unexpected(r.error())` to `result`.

Returns: `result`.

```
template<scannable-range<char> Range>
    vscan-result-type<Range> vscan(Range&& range, string_view fmt, scan_args
template<scannable-range<wchar_t> Range>
    vscan-result-type<Range> vscan(Range&& range, wstring_view fmt, wscan_arg
template<scannable-range<char> Range>
    vscan-result-type<Range> vscan(const locale& loc,
                                   Range&& range,
                                   string_view fmt,
                                   scan_args args);
template<scannable-range<wchar_t> Range>
    vscan-result-type<Range> vscan(const locale& loc,
                                   Range&& range,
                                   wstring_view fmt,
                                   wscan_args args);
```

Effects: Scans `range` for the character representations of scanning arguments provided by `args` scanned according to specifications given in `fmt`. If present, `loc` is used for locale-specific formatting. If successful, returns a `borrowed-tail-subrange-t` constructed from `it`

and `ranges::end(range)`, where `it` is an iterator pointing to the first character that was not scanned in `range`. Otherwise, returns a `scan_error` describing the error.

Throws: As specified in [scan.err].

Remarks: If `Range` is a reference to an array of `ranges::range_value_t<Range>`, `range` is treated as a NTCTS ([defns.ntcts]).

§ 7.2.7. Scanner [scan.scanner]

§ 7.2.7.1. Scanner requirements [scan.scanner.requirements]

A type `S` meets the *Scanner* requirements if it meets the

- *Cpp17DefaultConstructible*,
- *Cpp17CopyConstructible*,
- *Cpp17CopyAssignable*,
- *Cpp17Swappable*, and
- *Cpp17Destructible*,

requirements, and the expressions shown in [tab:scan.scanner] are valid and have the indicated semantics.

Given character type `charT`, source range type `Range`, and scanning argument type `T`, in [tab:scan.scanner]:

- `s` is a value of type (possibly const) `S`,
- `ls` is an lvalue of type `S`,
- `t` is an lvalue of type `T`,
- `PC` is `basic_scan_parse_context<charT>`,
- `SC` is `basic_scan_context<Range, charT>`,
- `pc` is an lvalue of type `PC`, and
- `sc` is an lvalue of type `FC`.

`pc.begin()` points to the beginning of the *scan-format-spec* ([scan.string]) of the replacement field being scanned in the format string. If *scan-format-spec* is not present or empty then either `pc.begin() == pc.end()` or `*pc.begin() == '}'`.

Expression	Return type	Requirement
<code>ls.parse(pc)</code>	<code>PC::iterator</code>	Parses <i>scan-format-spec</i> ([scan.string]) for type <code>T</code> in the range <code>[pc.begin(), pc.end())</code> until the first unmatched character. Throws <code>scan_format_string_error</code> unless the whole range is parsed or the unmatched character is <code>}</code> . Stores the parsed format specifiers in <code>*this</code> and returns an iterator past the end of the parsed range.
<code>s.scan(t, sc)</code>	<code>expected<SC::iterator, scan_error></code>	Scans <code>t</code> from <code>sc</code> according to the specifiers stored in <code>*this</code> . Reads the input from <code>sc.range()</code> or <code>sc.begin()</code> , and writes the result in <code>t</code> . On success, returns an iterator past the end of the last scanned character from <code>sc</code> , otherwise returns an object of type <code>scan_error</code> . The value of <code>t</code> after calling shall only depend on <code>sc.range()</code> , <code>sc.locale()</code> , and the range <code>[pc.begin(), pc.end())</code> from the last call to <code>s.parse(pc)</code> .

§ 7.2.7.2. Concept `scannable` [scan.scannable]

```

namespace std {
    template<class T, class Context,
            class Scanner = typename Context::template scanner_type<T>>
    concept scannable-with = // exposition only
        semiregular<Scanner> &&
        requires(Scanner& s, const Scanner& cs, T& t, Context& ctx,
                 basic_scan_parse_context<typename Context::char_type>& pctx
        {
            { s.parse(pctx) } -> same_as<typename decltype(pctx)::iterator>;
            { cs.scan(t, ctx) } -> same_as<expected<typename Context::iterator,
        };

```



```
template<class T, class charT>
    concept scannable =
        scannable-with<T, basic_scan_context<unspecified, charT>>;
}
```

A type `T` and a character type `charT` model `scannable` if `scanner<T, charT>` meets the *Scanner* requirements ([scan.scanner.requirements]).

[Note 1: `scannable<string_view, char>` is `true`, even though a `string_view` can only be scanned from a contiguous borrowed range. — end note]

§ 7.2.7.3. Scanner specializations

The functions defined in [scan.functions] use specializations of the class template `scanner` to scan individual arguments.

Let `charT` be either `char` or `wchar_t`. Each specialization of `scanner` is either enabled or disabled, as described below. A *debug-enabled* specialization of `scanner` additionally provides a public, constexpr, non-static member function `set_debug_format()` which modifies the state of the `scanner` to be as if the type of the *std-scan-format-spec* parsed by the last call to `parse` were `?`. Each header that declares the template `scanner` provides the following enabled specializations:

- The debug-enabled specializations

```
template<> struct scanner<char, char>;
template<> struct scanner<wchar_t, wchar_t>;
```

- For each `charT`, the debug-enabled string type specializations

```
template<class Allocator>
    struct scanner<basic_string<charT, char_traits<charT>, Allocator>,
template<> struct scanner<basic_string_view<charT>, charT>;
```

- For each `charT`, for each arithmetic type `ArithmeticT` other than `char`, `wchar_t`, `char8_t`, `char16_t`, or `char32_t`, a specialization

```
template<> struct scanner<ArithmeticT, charT>;
```

- For each `charT`, the pointer type specializations

```
template<> struct scanner<void*, charT>;
template<> struct scanner<const void*, charT>;
```

The `parse` member functions of these scanners interpret the format specification as a *std-scan-format-spec* as described in [scan.string.std].

For any types `T` and `charT` for which neither the library nor the user provides an explicit or partial specialization of the class template `scanner`, `scanner<T, charT>` is disabled.

If the library provides an explicit or partial specialization of `scanner<T, charT>`, that specialization is enabled and meets the *Scanner* requirements except as noted otherwise.

If `S` is a disabled specialization of `scanner`, these values are `false`:

- `is_default_constructible_v<S>`,
- `is_copy_constructible_v<S>`,
- `is_move_constructible_v<S>`,
- `is_copy_assignable_v<S>`, and
- `is_move_assignable_v<S>`.

An enabled specialization of `scanner<T, charT>` meets the *Scanner* requirements ([scan.scanner.requirements]).

§ 7.2.7.4. Class template `basic_scan_parse_context` [scan.parse.ctx]

```
namespace std {
    template<class charT>
    class basic_scan_parse_context {
    public:
        using char_type = charT;
        using const_iterator = typename basic_string_view<charT>::const_iterator;
        using iterator = const_iterator;

    private:
        iterator begin_;           // exposition only
        iterator end_;             // exposition only
    };
}
```

```

enum indexing { unknown, manual, automatic }; // exposition only
indexing indexing_;                          // exposition only
size_t next_arg_id_;                          // exposition only
size_t num_args_;                             // exposition only

public:
constexpr explicit basic_scan_parse_context(basic_string_view<charT> fr
basic_scan_parse_context(const basic_scan_parse_context&) = delete;
basic_scan_parse_context& operator=(const basic_scan_parse_context&) =

constexpr const_iterator begin() const noexcept { return begin_; }
constexpr const_iterator end() const noexcept { return end_; }
constexpr void advance_to(const_iterator it);

constexpr size_t next_arg_id();
constexpr void check_arg_id(size_t id);
};
}

```

An instance of `basic_scan_parse_context` holds the format string parsing state, consisting of the format string range being parsed and the argument counter for automatic indexing.

If a program declares an explicit or partial specialization of `basic_scan_parse_context`, the program is ill-formed, no diagnostic required.

```

constexpr explicit basic_scan_parse_context(basic_string_view<charT> fmt) :

```

Effects: Initializes `begin_` with `fmt.begin()`, `end_` with `fmt.end()`, `indexing_` with `unknown`, `next_arg_id_` with `0`, and `num_args_` with `0`.

[*Note 1:* Any call to `next_arg_id` or `check_arg_id` on an instance of `basic_scan_parse_context` initialized using this constructor is not a core constant expression. — *end note*]

```

constexpr void advance_to(const_iterator it);

```

Preconditions: `end()` is reachable from `it`.

Effects: Equivalent to: `begin_ = std::move(it)`.

```
constexpr size_t next_arg_id();
```

Effects: If `indexing != manual` is `true`, equivalent to:

```
if (indexing_ == unknown)
    indexing_ = automatic;
return next_arg_id_++;
```

Otherwise, the string is not a format string for `args`.

Remarks: Let `cur-arg-id` be the value of `next_arg_id_` prior to this call. Call expressions where `cur-arg-id >= num_args_` is `false` are not core constant expressions ([`expr.const`]).

```
constexpr void check_arg_id(size_t id);
```

Effects: If `indexing != automatic` is `true`, equivalent to:

```
if (indexing_ == unknown)
    indexing_ = manual;
```

Otherwise, the string is not a format string for `args`.

Remarks: A call to this function is a core constant expression ([`expr.const`]) only if `id < num_args_` is `true`.

§ 7.2.8. Class template `basic_scan_context` [`scan.context`]

```
namespace std {
    template<class Range, class charT>
    class basic_scan_context {
        iterator current_; // exposition only
        sentinel end_; // exposition only
        basic_scan_args<basic_scan_context> args_; // exposition only

    public:
```

```

using char_type = charT;
using range_type = Range;
using iterator = ranges::iterator_t<range_type>;
using sentinel = ranges::sentinel_t<range_type>;
template<class T> using scanner_type = scanner<T, char_type>;

basic_scan_arg<basic_scan_context> arg(size_t id) const noexcept;
std::locale locale();

iterator begin() const { return begin_; }
sentinel end() const { return end_; }
ranges::subrange<iterator, sentinel> range() const;
void advance_to(iterator it);
};
}

```

An instance of `basic_scan_context` holds scanning state consisting of the scanning arguments and the source range.

If a program declares an explicit or partial specialization of `basic_scan_context`, the program is ill-formed, no diagnostic required.

`Range` shall model `forward_range`, and its value type shall be `charT`. The iterator and sentinel types of `Range` shall model `copyable`.

`scan_context` is an alias for a specialization of `basic_scan_context` with a range type that can contain a reference to any other forward range with a value type of `char`. Similarly, `wscan_context` is an alias for a specialization of `basic_scan_context` with a range type that can contain a reference to any other forward range with a value type of `wchar_t`.

Recommended practice: For a given type `charT`, implementations should provide a single instantiation for reading from `basic_string<charT>`, `vector<charT>`, or any other container with contiguous storage by wrapping those in temporary objects with a uniform interface, such as a `span<charT>`.

```

basic_scan_arg<basic_scan_context> arg(size_t id) const noexcept;

```

Returns: `args_.get(id)`.

```

std::locale locale();

```

Returns: The locale passed to the scanning function if the latter takes one, and `std::locale()` otherwise.

```
ranges::subrange<iterator, sentinel> range() const;
```

Effects: Equivalent to: `return ranges::subrange(begin_, end_);`

```
void advance_to(iterator it) const;
```

Effects: Equivalent to: `begin_ = std::move(it);`

§ 7.2.9. Arguments [scan.arguments]

§ 7.2.9.1. Class template `basic_scan_arg` [scan.arg]

```
namespace std {
    template<class Context>
    class basic_scan_arg {
    public:
        class handle;

    private:
        using char_type = typename Context::char_type;           // exposition
        variant<
            monostate,
            signed char*, short*, int*, long*, long long*,
            unsigned char*, unsigned short*, unsigned int*, unsigned long*, unsigned long long*,
            bool*, char_type*, void**, const void**,
            float*, double*, long double*,
            basic_string<char_type>*, basic_string_view<char_type>*,
            handle> value;                                       // exposition
        template<class T> explicit basic_scan_arg(T& v) noexcept; // exposition

    public:
        basic_scan_arg() noexcept;

        explicit operator bool() const noexcept;
```

```

template<class Visitor>
    decltype(auto) visit(this basic_scan_arg arg, Visitor&& vis);
template<class R, class Visitor>
    R visit(this basic_scan_arg arg, Visitor&& vis);
};
}

```

An instance of `basic_scan_arg` provides access to a scanning argument for user-defined scanners.

The behavior of a program that adds specializations of `basic_scan_arg` is undefined.

```
basic_scan_arg() noexcept;
```

Postconditions: `!(*this)`.

```
template<class T> explicit basic_scan_arg(T& v) noexcept;
```

Constraints: `T` satisfies `formattable-with<Context>`.

Effects: Let `TD` be `remove_const_t<T>`.

- If `TD` is a standard signed integer type ([basic.fundamental]), a standard unsigned integer type, `bool`, `char-type`, `void*`, a standard floating-point type, `basic_string<char-type>`, or `basic_string_view<char-type>`, initializes `value` with `addressof(v)`;
- otherwise, initializes `value` with `handle(v)`.

```
explicit operator bool() const noexcept;
```

Returns: `!holds_alternative<monostate>(value)`.

```

template<class Visitor>
    decltype(auto) visit(this basic_scan_arg arg, Visitor&& vis);

```

Effects: Equivalent to: `return arg.value.visit(std::forward<Visitor>(vis));`

```
template<class R, class Visitor>
R visit(this basic_scan_arg arg, Visitor&& vis);
```

Effects: Equivalent to: `return arg.value.visit(std::forward<Visitor>(vis));`

The class `handle` allows scanning an object of a user-defined type.

```
namespace std {
    template<class Context>
    class basic_scan_arg<Context>::handle {
        void* ptr_; // exposition
        expected<void, scan_error> (*scan_)
            (basic_scan_parse_context<char_type>, Context&, void*); // exposition

        template<class T> explicit handle(T& val) noexcept; // exposition

        friend class basic_scan_arg<Context>; // exposition

    public:
        expected<void, scan_error>
            scan(basic_scan_parse_context<char_type>& parse_ctx, Context& ctx) const;
    };
}
```

```
template<class T> explicit handle(T& val) noexcept;
```

Mandates: `T` satisfies `scannable-with<Context>`.

Effects: Initializes `ptr_` with `addressof(val)` and `scan_` with

```
[] (basic_scan_parse_context<char_type>& parse_ctx, Context& scan_ctx,
    -> expected<void, scan_error> {
    typename Context::template scanner_type<T> s;
    auto p = do_parse(s, parse_ctx);
    if (!p) return unexpected(p.error());
    parse_ctx.advance_to(*p);
    auto r = s.scan(*static_cast<T*>(ptr_), scan_ctx);
    if (!r) return unexpected(r.error());
```



```

scan_ctx.advance_to(*r);
return {};
}

```

where `do-parse(s, pc)`:

- has a return type of `expected<basic_scan_parse_context<char_type>::iterator, scan_error>`,
- calls `s.parse(pc)`,
- catches exceptions derived from `scan_format_string_error` thrown by `s.parse`. If such an exception is caught, returns a `scan_error` with a code of `invalid_format_string`.
- Otherwise, returns the iterator returned by `s.parse`.

```

expected<void, scan_error> scan(basic_scan_parse_context<char_type>& parse_

```

Effects: Equivalent to: `return scan_(parse_ctx, scan_ctx, ptr_);`

§ 7.2.9.2. Class template `scan-arg-store` [`scan.arg.store`]

```

namespace std {
    template<class Context, class... Args>
    class scan-arg-store { // exposition on
        array<basic_scan_arg<Context>, sizeof...(Args)> args; // exposition on
    };
}

```

An instance of `format-arg-store` stores scanning arguments.

```

template<class Context = scan_context, class... Args>
constexpr scan-arg-store<Context, Args...>
    make_scan_args(std::tuple& values);

```

Preconditions: The type `typename Context::template scanner_type<Ti>` meets the *Scanner* requirements ([`scan.scanner.requirements`]) for each `Ti` in `Args`.

Returns: An object of type `scan-arg-store<Context, Args...>`. All elements of the `data` member of the returned object are initialized with `basic_scan_arg<Context>(get<i>(values))`, where `i` is an index in the range of `[0, sizeof...(Args))`.

```
template<class... Args>
constexpr scan-arg-store<wscan_context, Args...>
    make_wscan_args(std::tuple& values);
```

Effects: Equivalent to: `return make_scan_args<wscan_context>(values)`.

§ 7.2.9.3. Class template `basic_scan_args` [`scan.args`]

```
namespace std {
    template<class Context>
    class basic_scan_args {
        size_t size_; // exposition only
        const basic_scan_arg<Context>* data_; // exposition only

    public:
        basic_scan_args() noexcept;

        template<class... Args>
            basic_scan_args(const scan-arg-store<Context, Args...>& store) noexcept;

        basic_scan_arg<Context> get(size_t i) noexcept;
    };

    template<class Context, class... Args>
        basic_scan_args(scan-arg-store<Context, Args...>) -> basic_scan_args<Context, Args...>
}
```

An instance of `basic_scan_args` provides access to scanning arguments. Implementations should optimize the representation of `basic_scan_args` for a small number of scanning arguments.

[*Note 1:* For example, by storing indices of type alternatives separately from values and packing the former. — *end note*]

```
template<class... Args>
    basic_scan_args(const scan-arg-store<Context, Args...>& store) noexcept;
```

Effects: Initializes `size_` with `sizeof...(Args)` and `data_` with `store.args.data()`;

```
basic_scan_arg<Context> get(size_t i) noexcept;
```

Returns: `i < size_ ? data_[i] : basic_scan_arg<Context>()`.

§ References

§ Informative References

[ATTR]

Common Function Attributes. URL: <https://gcc.gnu.org/onlinedocs/gcc-8.2.0/gcc/Common-Function-Attributes.html>

[CODESEARCH]

Andrew Tomazos. *Code search engine website*. URL: <https://codesearch.isocpp.org>

[FMT]

Victor Zverovich et al. *The fmt library*. URL: <https://github.com/fmtlib/fmt>

[N4412]

Jens Maurer. *N4412: Shortcomings of iostreams*. URL: <http://open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4412.html>

[N4988]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 5 August 2024. URL: <https://wg21.link/n4988>

[P0355]

Howard E. Hinnant; Tomasz Kamiński. *Extending <chrono> to Calendars and Time Zones*. URL: <https://wg21.link/p0355>

[P0645]

Victor Zverovich. *Text Formatting*. URL: <https://wg21.link/p0645>

[P1361]

Victor Zverovich; Daniela Engert; Howard E. Hinnant. *Integration of chrono with text formatting*. URL: <https://wg21.link/p1361>

[P2286]

Barry Revzin. *Formatting Ranges*. URL: <https://wg21.link/p2286>

[P2561]

Barry Revzin. *A control flow operator*. URL: <https://wg21.link/p2561>

[P2637]

Barry Revzin. *Member `visit`*. URL: <https://wg21.link/p2637>

[PARSE]

Python `parse` package. URL: <https://pypi.org/project/parse/>

[SCNLIB]

Elias Kosunen. *scnlib: scanf for modern C++*. URL: <https://github.com/eliaskosunen/scnlib>